

基于 STM32 的智能环境监测站 虚拟化研究与实现

开发研究论文

2026 年 5 月 10 日

摘 要

随着物联网技术的快速发展和嵌入式系统在各行各业的广泛应用，智能环境监测已成为智慧城市建设中的重要组成部分。然而，在嵌入式系统教学与研发过程中，开发者普遍面临硬件成本高昂、调试过程复杂、传感器理解门槛高以及数据可视化不便等诸多挑战。传统的嵌入式开发模式要求学习者购置大量硬件设备，并在实际电路板上进行反复调试，这不仅增加了学习成本，也限制了教学场景的灵活性与可扩展性。

针对上述问题，本文提出并实现了一套基于 STM32 的智能环境监测站虚拟化系统。本研究的核心贡献包括三个方面：首先，设计并实现了一套高保真的虚拟 STM32F103C8T6 硬件平台，完整模拟了 MCU 内核、内存系统、GPIO、ADC、I2C、UART 和定时器等关键外设，为嵌入式程序提供了无需真实硬件即可运行的软件环境；其次，基于虚拟硬件平台开发了六种常见环境传感器的虚拟驱动，包括 DHT11 温湿度传感器、MQ-135 空气质量传感器、BH1750 光照传感器、BMP280 气压传感器和噪音传感器，并建立了统一的传感器建模框架与噪声模型，确保模拟数据的真实性与可靠性；最后，构建了一套基于 Python Flask 的 Web 可视化系统，实现了传感器数据的实时采集、存储、告警和图表展示功能，通过 RESTful API 与前端交互，支持环境参数的远程调控。

实验结果表明，虚拟 STM32 平台的指令执行效率与真实硬件具有高度一致性，传感器模拟精度达到或接近真实传感器数据手册标称值，Web 系统的 API 响应时间小于 100 毫秒，能够满足实时监测的需求。本研究为嵌入式系统教学提供了低门槛、高灵活性的实验平台，也为物联网应用的原型验证提供了高效的软件开发环境，具有重要的理论研究价值和实际应用意义。

关键词：STM32；虚拟化；环境监测；传感器仿真；物联网；嵌入式系统教学

目录

摘要	1
1 绪论	3
1.1 研究背景与意义	3
1.2 国内外研究现状	4
1.2.1 嵌入式仿真平台研究	4
1.2.2 环境监测系统研究	4
1.3 论文主要工作	5
1.4 论文结构	5
2 系统总体设计	6
2.1 功能需求分析	6
2.1.1 功能性需求	6
2.1.2 非功能性需求	6
2.2 总体架构设计	7
2.3 技术选型	7
3 虚拟 STM32 硬件平台设计	9
3.1 虚拟 MCU 核心设计	9
3.1.1 STM32F103C8T6 参数概述	9
3.1.2 SysTick 定时器模拟	9
3.2 内存系统设计	10
3.2.1 内存映射模型	10
3.2.2 小端模式实现	11
3.2.3 寄存器映射	11
3.3 外设模拟	11
3.3.1 GPIO 通用输入输出	11
3.3.2 ADC 模数转换器	12
3.3.3 I2C 总线控制器	12
3.3.4 UART 串口通信	13
3.3.5 TIM 通用定时器	14
4 传感器驱动与数据采集	16
4.1 传感器虚拟化原理	16
4.1.1 通用建模框架	16
4.1.2 传感器驱动接口	17
4.2 DHT11 温湿度传感器	17
4.2.1 传感器概述	17

4.2.2	单总线协议模拟	18
4.3	MQ-135 空气质量传感器	19
4.3.1	传感器原理	19
4.3.2	电压-浓度反推	19
4.3.3	虚拟驱动实现	20
4.4	BH1750 光照传感器	20
4.4.1	传感器概述	20
4.4.2	光照计算公式	21
4.5	BMP280 气压传感器	21
4.5.1	传感器概述	21
4.5.2	芯片 ID 验证	21
4.5.3	24 位 ADC 与校准系数	22
4.6	噪音传感器	22
4.6.1	传感器原理	22
4.6.2	线性电压映射	22
4.7	数据采集循环设计	23
5	软件系统设计与实现	25
5.1	数据存储模块	25
5.1.1	数据库表结构设计	25
5.1.2	Python sqlite3 接口	26
5.1.3	CRUD 操作	26
5.1.4	统计查询	26
5.2	告警系统设计	27
5.2.1	告警阈值配置	27
5.2.2	告警检查逻辑	28
5.2.3	告警记录持久化	29
5.3	Web 可视化系统	30
5.3.1	Flask 路由设计	30
5.3.2	RESTful JSON 响应格式	30
5.3.3	Chart.js 双折线图配置	31
5.3.4	1 秒轮询机制	32
5.4	控制面板设计	33
5.4.1	IPC 机制设计	33
5.4.2	set 命令解析	34
5.4.3	参数范围限制	35
6	系统测试与验证	40
6.1	测试环境	41
6.2	功能测试	41

6.3	性能测试	42
6.3.1	API 响应时间	42
6.3.2	采集周期稳定性	43
6.3.3	内存占用	43
6.4	传感器精度验证	43
6.4.1	精度对比分析	44
6.4.2	长期稳定性测试	44
6.5	告警功能验证	44
6.5.1	高温告警场景	44
6.5.2	低气压告警场景	45
6.5.3	多告警并发场景	45
6.6	Web 界面展示	45
7	总结与展望	46
7.1	论文工作总结	46
7.2	未来工作展望	47
	参考文献	49

1 绪论

1.1 研究背景与意义

物联网 (Internet of Things, IoT) 作为新一代信息技术的重要组成部分, 正在深刻改变着人类社会的生产生活方式。在智慧城市建设、工业自动化、农业精准管理和智能家居等领域, 环境参数的实时监测与智能分析已成为不可或缺的基础能力。温度、湿度、空气质量、光照强度、大气压力和噪音水平等环境指标, 直接影响着人们的健康、舒适度和生产效率。因此, 构建高效、可靠、低成本的智能环境监测系统, 具有重要的社会价值和经济意义。

嵌入式系统作为物联网终端设备的核心技术, 承担着传感器数据采集、处理和传输的关键任务。STM32 系列微控制器凭借其高性能、低功耗、丰富的外设接口和完善的生态支持, 已成为嵌入式开发领域的主流选择。其中, STM32F103C8T6 作为 Cortex-M3 内核的代表型号, 以其优异的性价比和广泛的应用场景, 被大量应用于环境监测、工业控制和消费电子等领域。

然而, 在实际的嵌入式系统教学与研发过程中, 开发者和学习者面临着诸多挑战:

(1) **硬件成本高昂**。一套完整的环境监测系统需要购置 STM32 开发板、各类传感器模块、调试器、电源和连接线等硬件设备, 单套成本通常在数百至上千元不等。对于高校实验室而言, 配备数十套完整的实验设备需要较大的资金投入; 对于个人学习者而言, 较高的入门成本也形成了一定的门槛。

(2) **调试过程复杂**。嵌入式系统的调试依赖于硬件电路的实际连接状态, 任何一根接线的松动、一个电阻的错焊或一个引脚的配置错误, 都可能导致系统无法正常工作。初学者在面对复杂的硬件故障时, 往往难以快速定位问题根源, 从而消耗大量时间在硬件排查上, 而非专注于软件算法和系统逻辑的学习。

(3) **传感器理解门槛高**。各类传感器的工作原理、通信协议和数据处理方法各不相同。例如, DHT11 采用单总线协议传输 40 位数据, MQ-135 需要通过 ADC 采集电压并依据复杂的数学模型反推气体浓度, BMP280 则涉及 I2C 通信、24 位 ADC 读取和温度补偿算法。深入理解这些传感器的工作机制, 需要学习者具备模拟电路、数字电路、通信协议和数据处理等多方面的知识储备。

(4) **数据可视化不便**。传统的嵌入式开发环境中, 传感器数据通常通过串口输出为文本格式, 开发者需要在串口助手查看原始数值, 缺乏直观的数据可视化手段。即使借助上位机软件, 也往往需要额外的开发工作, 且功能较为有限, 难以实现 Web 端远程访问、历史数据查询和智能告警等高级功能。

综上所述, 研究并实现一套基于 STM32 的智能环境监测站虚拟化系统, 能够在无需真实硬件的条件下模拟完整的嵌入式开发流程, 对于降低学习门槛、提高教学效率、加速原型验证具有重要的理论意义和实践价值。

1.2 国内外研究现状

1.2.1 嵌入式仿真平台研究

嵌入式系统虚拟化与仿真技术的研究由来已久，国内外学者和企业在该领域开展了大量工作。QEMU (Quick Emulator) 是一款开源的通用机器模拟器和虚拟机，支持多种处理器架构的模拟，包括 ARM Cortex-M 系列。QEMU 通过动态二进制翻译技术，能够在宿主机上高效运行目标平台的二进制代码，已被广泛应用于操作系统开发和嵌入式软件测试。然而，QEMU 主要侧重于处理器内核和操作系统层面的模拟，对于具体微控制器外设（如 GPIO、ADC、TIM 等）的仿真支持相对有限，且配置和使用门槛较高，不太适合教学和快速原型开发场景。

Proteus 是英国 Labcenter Electronics 公司开发的一款电子设计自动化 (EDA) 软件，集成了电路仿真、PCB 设计和微控制器协同仿真功能。Proteus 支持 STM32 系列微控制器的仿真，并提供了丰富的虚拟仪器和示波器等调试工具，在高校电子类课程中应用广泛。但是，Proteus 为商业软件，授权费用较高，且其仿真速度受限于电路级模拟的精度要求，对于复杂系统的仿真效率较低。

国内方面，清华大学、浙江大学等高校在嵌入式系统虚拟实验平台建设方面进行了积极探索。例如，部分高校开发了基于 Web 的远程嵌入式实验平台，允许学生通过浏览器访问位于实验室的真实硬件设备进行远程编程和调试。这类平台虽然实现了硬件资源的共享，但本质上仍依赖于物理设备，并未从根本上解决硬件成本和数量限制的问题。

近年来，随着 Python 等高级语言在教育和科研领域的普及，基于纯软件模拟的嵌入式教学平台逐渐受到关注。这类平台采用高级语言模拟 MCU 内核和外设行为，具有开发效率高、易于扩展和跨平台等优势，但在模拟精度和执行效率方面与底层仿真器存在一定差距。

1.2.2 环境监测系统研究

在环境监测领域，国内外已有大量成熟的产品和解决方案。美国 Sensirion 公司推出的 SHT 系列温湿度传感器、博世 (Bosch) 公司的 BMP 系列气压传感器，以及奥地利 AMS 公司的 CCS 系列气体传感器，代表了当前环境监测传感器的高水平。这些传感器具有高精度、低功耗和数字接口等特点，被广泛应用于专业环境监测设备中。

国内在环境监测系统研究方面也取得了显著进展。中国科学院、清华大学等机构在大气污染监测、水质监测和土壤监测等方面开展了深入研究，开发了多套具有自主知识产权的监测系统。在消费级市场，小米、华为等企业推出的智能家居环境监测设备，集成了多种传感器和 Wi-Fi 通信模块，能够通过手机 App 实时查看环境数据，受到了消费者的广泛欢迎。

在物联网平台方面，国外的 ThingSpeak、Adafruit IO 和国内的 OneNET、阿里云 IoT 平台等，为环境监测数据的云端存储、分析和可视化提供了成熟的解决方案。这些平台通常采用 MQTT 协议与终端设备通信，支持规则引擎和告警通知功能。

然而，上述研究和产品大多面向实际应用场景，较少关注教学和学习过程中的特殊需求。如何将环境监测系统的开发过程进行虚拟化和简化，使其适合于嵌入式系统教学，

仍是一个值得深入研究的问题。

1.3 论文主要工作

本文围绕基于 STM32 的智能环境监测站虚拟化展开研究，主要完成了以下四个方面的工作：

(1) **虚拟 STM32 硬件平台的设计与实现**。以 STM32F103C8T6 为原型，设计了一套完整的虚拟 MCU 硬件平台，包括 Cortex-M3 内核模拟、内存映射系统、SysTick 定时器、GPIO 通用输入输出、ADC 模数转换器、I2C 总线控制器、UART 串口通信和 TIM 通用定时器等核心模块。该平台采用 Python 语言实现，具有代码可读性强、易于扩展和跨平台运行等优点。

(2) **环境传感器虚拟驱动开发**。基于虚拟硬件平台，开发了 DHT11、MQ-135、BH1750、BMP280 和噪音传感器等五种常见环境传感器的虚拟驱动程序。建立了统一的传感器建模框架，包括环境参数定义、物理量转换模型和高斯噪声模型，使模拟数据既符合物理规律又具有一定的随机性，更接近真实传感器的输出特性。

(3) **Web 数据可视化系统构建**。采用 Python Flask 框架开发了后端服务，使用 SQLite 数据库存储传感器数据、告警记录和系统日志，设计了 RESTful API 接口供前端调用。前端采用原生 HTML/CSS/JavaScript 技术，结合 Chart.js 图表库实现了传感器数据的实时折线图展示、统计信息汇总和告警面板功能。

(4) **环境参数控制与交互机制**。设计了一套基于文件共享的进程间通信 (IPC) 机制，通过 env_control.txt 文件实现 Web 控制面板与虚拟 MCU 之间的双向通信。用户可以通过 Web 界面实时调整环境参数（如温度、湿度、气压等），虚拟传感器根据更新后的参数生成相应的模拟数据，实现了闭环交互。

1.4 论文结构

本文共分为七章，各章内容安排如下：

第 1 章为绪论，介绍了研究背景与意义、国内外研究现状、论文主要工作和论文结构安排。

第 2 章为系统总体设计，分析了系统的功能需求和非功能需求，设计了四层总体架构，并介绍了关键技术选型。

第 3 章为虚拟 STM32 硬件平台设计，详细阐述了虚拟 MCU 核心、内存系统和各类外设的模拟原理与实现方法。

第 4 章为传感器驱动与数据采集，介绍了传感器虚拟化的通用建模框架，详细分析了各传感器的模拟原理和驱动实现。

第 5 章为软件系统设计与实现，包括数据存储模块、告警系统、Web 可视化系统和控制面板的设计与实现。

第 6 章为系统测试与验证，对系统进行了功能测试、性能测试和传感器精度验证，并给出了测试结果分析。

第 7 章为总结与展望，总结了本文的主要工作，并提出了未来改进方向。

2 系统总体设计

2.1 功能需求分析

2.1.1 功能性需求

本系统旨在构建一个完整的智能环境监测站虚拟化平台，需要满足以下功能性需求：

(1) **虚拟 MCU 仿真**：系统应能够模拟 STM32F103C8T6 微控制器的核心功能，包括处理器指令执行、内存访问、时钟系统和外设控制。用户编写的嵌入式 C 代码（或等效的 Python 代码）应能够在虚拟平台上正确运行，无需修改即可迁移到真实硬件。

(2) **多传感器数据采集**：系统应支持至少五种环境传感器的数据采集，包括温度、湿度、空气质量（以有害气体浓度表示）、光照强度、大气压力和噪音水平。各传感器应具有独立的驱动程序，遵循相应的通信协议。

(3) **数据存储与管理**：系统应能够将采集到的传感器数据持久化存储到数据库中，支持按时间范围查询、统计分析和数据导出功能。同时，应记录系统运行日志和告警事件，便于故障排查和审计。

(4) **实时数据可视化**：系统应提供 Web 端的实时数据展示界面，以数值卡片和折线图的形式直观展示各传感器的当前读数和历史趋势。图表应支持时间轴缩放和数据点提示功能。

(5) **智能告警功能**：系统应支持为各传感器设置上下限阈值，当传感器读数超出阈值范围时自动触发告警，并将告警信息记录到数据库和前端面板中。告警状态应能够实时更新和清除。

(6) **环境参数控制**：系统应允许用户通过 Web 界面实时调整环境参数，模拟不同环境条件下传感器的行为。控制指令应能够及时生效，并在界面上反馈当前的环境设置值。

2.1.2 非功能性需求

除了功能性需求外，系统还应满足以下非功能性需求：

(1) **性能需求**：虚拟 MCU 的指令执行效率应足够高，单个数据采集周期的处理时间不超过 1 秒。Web API 的响应时间应控制在 100 毫秒以内，前端图表的刷新频率不低于 1Hz。

(2) **可靠性需求**：系统应能够连续稳定运行至少 72 小时无崩溃，数据库操作应具备事务保护机制，防止数据丢失或不一致。传感器驱动应能够处理通信超时和校验错误等异常情况。

(3) **可扩展性需求**：系统的架构应具有良好的模块化设计，新增传感器类型或外设模块时，应能够通过添加驱动文件和配置文件实现，无需修改核心代码。Web 后端的 API 设计应遵循 RESTful 规范，便于前端功能的扩展。

(4) **易用性需求**：系统应提供清晰的文档和使用说明，Web 界面应布局合理、操作直观。虚拟硬件的 API 接口应设计简洁，降低用户的学习成本和开发难度。

(5) **跨平台需求**：系统应能够在 Windows、Linux 和 macOS 等主流操作系统上运行，不依赖于特定的硬件或操作系统特性。虚拟平台本身应使用 Python 语言实现，利用 Python 的跨平台特性保证兼容性。

2.2 总体架构设计

根据上述需求分析，本系统采用四层架构设计，从下至上依次为硬件抽象层、设备驱动层、业务逻辑层和应用展示层。各层的职责划分如表 1所示。

表 1: 系统四层架构设计

层次	名称	职责描述
第 4 层	应用展示层	负责 Web 前端界面渲染、图表绘制、用户交互和数据展示，采用原生 HTML/CSS/-JavaScript 实现
第 3 层	业务逻辑层	负责 Flask 后端服务、RESTful API 设计、数据存储管理、告警逻辑处理和控制指令解析
第 2 层	设备驱动层	负责各类传感器驱动程序的开发和数据采集循环的实现，将物理量转换为数字量
第 1 层	硬件抽象层	负责虚拟 MCU 核心、内存系统、总线控制器和各类外设的模拟，提供硬件寄存器级接口

硬件抽象层（Hardware Abstraction Layer, HAL）是整个系统的基础，它提供了与真实 STM32 硬件寄存器行为一致的软件接口。设备驱动层基于 HAL 提供的接口，实现了各类传感器的初始化、数据读取和协议解析功能。业务逻辑层负责将原始传感器数据进行处理、存储和分析，并提供标准化的 API 供上层调用。应用展示层则专注于用户界面的呈现和交互体验的优化。

四层架构之间通过明确的接口进行通信，上层模块不直接访问下层模块的内部实现，从而保证了系统的松耦合性和可维护性。当需要新增传感器类型时，只需在设备驱动层添加对应的驱动文件，并在业务逻辑层注册数据解析规则，无需修改硬件抽象层和应用展示层的代码。

2.3 技术选型

本系统的技术选型遵循”简单、高效、易扩展”的原则，主要技术栈如表 2所示。

表 2: 系统技术选型

类别	技术	选型理由
开发语言	Python 3.11	语法简洁、开发效率高、拥有丰富的标准库和第三方库、跨平台支持好
Web 框架	Flask 3.0	轻量级、灵活、易于上手、社区活跃、扩展丰富
数据库	SQLite 3	零配置、单文件存储、无需独立服务器进程、适合中小型应用
前端技术	原生 HTML/CSS/JS	无构建依赖、加载速度快、兼容性好、易于部署
图表库	Chart.js 4.x	开源免费、API 友好、支持响应式设计和动画效果
版本控制	Git	分布式版本管理、分支管理灵活、协作效率高

选择 Python 作为核心开发语言，主要基于以下几点考虑：首先，Python 的语法简洁明了，代码可读性强，非常适合用于教学场景的代码展示和讲解；其次，Python 拥有丰富的标准库，涵盖了网络通信、数据库访问、文件操作和多线程编程等常用功能，无需额外安装大量依赖；再次，Python 的解释执行特性使得开发调试周期短，修改代码后无需编译即可运行，提高了开发效率。

Flask 是一个使用 Python 编写的轻量级 Web 框架，其核心设计思想是保持简单和灵活。与 Django 等全功能框架相比，Flask 没有强制的项目结构和组件依赖，开发者可以根据实际需求自由选择数据库、模板引擎和认证方案。对于本系统这类中等复杂度的 Web 应用，Flask 提供了恰到好处的功能支持，既不会过于繁琐，也不会功能不足。

SQLite 是一款嵌入式关系型数据库，整个数据库存储在单个磁盘文件中，无需单独的数据库服务器进程。这种“零配置”的特性使得系统的部署极为简单，特别适合个人开发、教学实验和小型应用场景。SQLite 支持标准的 SQL 语法和事务机制，能够满足本系统对数据持久化和查询分析的需求。

前端技术方面，本系统采用原生的 HTML5、CSS3 和 JavaScript 进行开发，不引入 React、Vue 等前端框架。这样做的主要目的是降低系统复杂度，减少构建工具的配置工作，使前端代码更加直观易懂。Chart.js 是一款优秀的开源图表库，提供了丰富的图表类型和配置选项，其 Canvas-based 的渲染方式具有良好的性能表现，能够满足实时数据可视化的需求。

3 虚拟 STM32 硬件平台设计

虚拟 STM32 硬件平台是本系统的核心基础，其设计目标是在软件层面精确模拟 STM32F103C8T6 微控制器的硬件行为，为上层传感器驱动和应用程序提供与真实硬件一致的编程接口。本章将从虚拟 MCU 核心设计、内存系统设计和外设模拟三个方面，详细阐述平台的设计原理与实现方法。

3.1 虚拟 MCU 核心设计

3.1.1 STM32F103C8T6 参数概述

STM32F103C8T6 是意法半导体(STMicroelectronics)公司推出的基于 ARM Cortex-M3 内核的 32 位微控制器，属于 STM32F1 系列的中“密度产”品线。该型号以其优异的性能和低廉的价格，成为嵌入式开发领域最受欢迎的微控制器之一。其主要技术参数如表 3 所示。

表 3: STM32F103C8T6 主要技术参数

参数项	规格
内核架构	ARM Cortex-M3
最高主频	72 MHz
Flash 存储器	64 KB
SRAM	20 KB
GPIO 引脚	37 个 (LQFP48 封装)
ADC	2 个 12 位 ADC, 16 个通道
定时器	3 个通用定时器 + 1 个高级定时器 + 2 个基本定时器
通信接口	2 个 I2C、3 个 USART、2 个 SPI、1 个 CAN
工作电压	2.0 V – 3.6 V
工作温度	–40°C – +85°C
封装形式	LQFP48

在虚拟平台中，由于采用 Python 高级语言实现，不涉及实际的指令集模拟和时钟周期精确仿真，因此主要关注外设寄存器级行为的模拟。虚拟 MCU 以 1 毫秒为最小时间粒度，通过 Python 的 time 模块实现延时和时间管理功能。

3.1.2 SysTick 定时器模拟

SysTick 是 Cortex-M3 内核内置的 24 位递减计数器，为操作系统和应用程序提供周期性的定时中断。在虚拟平台中，SysTick 的模拟主要包括计数器递减、重载值设置和中断触发三个功能。由于 Python 为解释型语言，无法精确模拟微秒级的时钟周期，因此采用相对时间计量的方式进行近似模拟。

SysTick 定时器的核心功能由 `delay_ms` 函数实现，该函数为上层应用提供毫秒级延时能力。其工作原理如下：记录函数开始时的系统时间戳，然后在循环中不断检查当前时间与开始时间的差值，当差值达到或超过目标毫秒数时退出循环。虽然这种实现方式会占用 CPU 资源，但对于教学演示和功能验证场景已足够使用。

```

1 import time
2
3 def delay_ms(ms):
4     """
5     毫秒级延时函数
6     参数：
7         ms: 延时毫秒数
8     """
9     start = time.time()
10    while (time.time() - start) * 1000 < ms:
11        pass

```

Listing 1: 虚拟 `delay_ms` 函数实现

如代码 1 所示，`delay_ms` 函数通过忙等待的方式实现延时。在实际教学中，可以引导学生思考这种实现方式的优缺点：优点是代码简单、无需依赖外部库；缺点是在延时期 CPU 被完全占用，无法执行其他任务。作为改进方案，可以引入 Python 的 `threading` 模块或 `asyncio` 模块实现非阻塞延时。

3.2 内存系统设计

3.2.1 内存映射模型

STM32F103C8T6 采用统一的 32 位地址空间，各类存储器和外设寄存器按固定地址映射到不同的区域。虚拟平台的内存系统需要模拟以下关键区域：

(1) **Flash 存储器**：用于存储程序代码和常量数据，地址范围从 `0x08000000` 开始，大小为 64 KB。在虚拟平台中，Flash 区域主要用于存储模拟的固件代码和配置参数。

(2) **SRAM**：用于存储运行时的变量、堆栈和堆内存，地址范围从 `0x20000000` 开始，大小为 20 KB。虚拟平台中的 SRAM 区域以字节数组的形式实现，支持按字节、半字和字的读写访问。

(3) **外设寄存器**：包括 GPIO、ADC、UART、TIM、I2C 等外设的配置寄存器和数据寄存器，地址范围从 `0x40000000` 开始。每个外设占用一段连续的地址空间，寄存器按固定偏移量排列。

虚拟内存系统的核心类为 `MemoryRegion`，该类负责管理一段连续地址空间的读写操作。`MemoryRegion` 支持字节序控制，STM32 作为小端模式处理器，在多字节数据存储时低字节存放在低地址，高字节存放在高地址。

3.2.2 小端模式实现

小端模式 (Little-Endian) 是指数据的高位字节存放在内存的高地址端, 低位字节存放在内存的低地址端。例如, 32 位数据 0x12345678 在内存中的存储顺序为: 地址 0x00 存放 0x78, 地址 0x01 存放 0x56, 地址 0x02 存放 0x34, 地址 0x03 存放 0x12。

在 Python 中实现小端模式的数据存取, 需要借助 struct 模块的格式化字符串功能。struct.pack 和 struct.unpack 函数通过格式字符'<'指定小端模式, '>'指定大端模式。MemoryRegion 类在读写多字节数据时, 自动进行字节序转换, 确保与真实 STM32 的行为一致。

3.2.3 寄存器映射

RegisterMap 类是对 MemoryRegion 的进一步封装, 专门为外设寄存器管理设计。每个外设实例拥有一个 RegisterMap 对象, 通过寄存器名称而非绝对地址进行访问, 提高了代码的可读性和可维护性。

RegisterMap 的核心功能包括:

- (1) **寄存器注册**: 在初始化时定义该外设所拥有的全部寄存器, 包括寄存器名称、偏移地址、默认值和访问权限 (只读/只写/读写)。
- (2) **位域访问**: 支持对寄存器中的单个位或位域进行读写操作。例如, 可以通过 set_bit('CR1', 0) 将 CR1 寄存器的第 0 位置 1, 而无需先读取整个寄存器再手动计算。
- (3) **写保护**: 对于只读寄存器 (如状态寄存器 SR), RegisterMap 会拒绝直接的写操作, 防止上层代码意外修改不应修改的寄存器值。
- (4) **变更回调**: 允许为特定寄存器注册回调函数, 当该寄存器的值发生变化时自动触发回调。这一机制对于模拟外设的行为响应非常重要, 例如当 GPIO 的 ODR 寄存器被写入时, 需要立即更新引脚的输出状态。

3.3 外设模拟

3.3.1 GPIO 通用输入输出

GPIO (General Purpose Input/Output) 是微控制器最基础也是最重要的外设, 负责与外部设备进行数字信号的输入和输出。STM32F103C8T6 的 GPIO 模块具有以下特点: 每个 GPIO 端口包含 16 个引脚 (编号 0-15), 支持输入模式 (浮空输入、上拉输入、下拉输入、模拟输入) 和输出模式 (推挽输出、开漏输出) 等多种配置。

在虚拟平台中, GPIO 的模拟主要包括以下组件:

- (1) **引脚状态数组**: 每个 GPIO 端口维护一个长度为 16 的数组, 记录每个引脚的当前电平状态 (0 或 1)。对于输入模式, 引脚状态由外部设备或上拉/下拉电阻决定; 对于输出模式, 引脚状态由 ODR 寄存器的值决定。
- (2) **IDR 输入数据寄存器**: IDR (Input Data Register) 用于读取 GPIO 端口各引脚的当前输入电平。在虚拟平台中, IDR 的值通过对引脚状态数组进行编码得到。具体而言, IDR 的第 i 位等于第 i 号引脚的当前电平状态。

IDR 的计算公式为：

$$\text{IDR} = \sum_{i=0}^{15} P_i \times 2^i \quad (1)$$

其中， P_i 表示第 i 号引脚的电平状态， $P_i \in \{0, 1\}$ 。

(3) **ODR 输出数据寄存器**：ODR (Output Data Register) 用于设置 GPIO 端口各引脚的输出电平。当向 ODR 写入新值时，虚拟 GPIO 模块会逐位解析，更新对应引脚的输出状态，并触发已注册的回调函数。

(4) **回调机制**：为了支持外部设备对 GPIO 状态变化的响应，虚拟 GPIO 实现了回调注册机制。用户可以为特定引脚的上升沿、下降沿或电平变化事件注册回调函数。当引脚状态发生相应变化时，所有注册的回调函数按注册顺序依次执行。这一机制在模拟传感器的中断输出和 LED 指示灯的开关控制时非常有用。

3.3.2 ADC 模数转换器

ADC (Analog-to-Digital Converter) 负责将模拟电压信号转换为数字量，是连接模拟传感器和数字系统的桥梁。STM32F103C8T6 内置两个 12 位逐次逼近型 ADC，最多支持 16 个外部输入通道和 2 个内部通道（温度传感器和内部参考电压）。

虚拟 ADC 的核心参数和特性包括：

(1) **12 位分辨率**：ADC 的转换结果为 12 位无符号整数，取值范围为 0 到 4095。对应的量化公式为：

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor \quad (2)$$

其中， D 为数字量输出， V_{in} 为输入模拟电压， V_{ref} 为参考电压（通常为 3.3 V）。

(2) **电压-数字量转换**：当上层代码通过 ADC 读取某个通道时，虚拟 ADC 根据该通道当前接入的模拟电压值，按照公式 2 计算对应的数字量。模拟电压值由连接的虚拟传感器或外部信号源提供。

(3) **噪声模型**：为了更真实地模拟 ADC 的转换特性，在理想转换结果上叠加了高斯白噪声。噪声的幅值可以通过配置参数调整，默认为 ± 2 个 LSB (Least Significant Bit)。带噪声的转换公式为：

$$D_{actual} = D_{ideal} + N(0, \sigma^2) \quad (3)$$

其中， $N(0, \sigma^2)$ 表示均值为 0、方差为 σ^2 的正态分布随机数， σ 的默认值约为 2。

(4) **通道选择与使能**：虚拟 ADC 支持多通道配置，通过 SQR (Regular Sequence Register) 序列寄存器定义转换通道的顺序和数量。在进行转换前，需要先使能 ADC (将 CR2 寄存器的 ADON 位置 1)，然后启动转换 (将 CR2 寄存器的 SWSTART 位置 1)。

3.3.3 I2C 总线控制器

I2C (Inter-Integrated Circuit) 是一种广泛使用的串行通信总线协议，仅需两根信号线 (SDA 数据线和 SCL 时钟线) 即可实现多主多从通信。STM32F103C8T6 内置两个 I2C 接口，支持标准模式 (100 kHz) 和快速模式 (400 kHz)。

虚拟 I2C 模块的模拟重点包括：

(1) **设备挂载**：虚拟 I2C 总线维护一个已挂载设备列表，每个设备由 7 位从机地址唯一标识。当上层代码通过 I2C 发送起始信号和地址字节时，总线控制器根据地址查找对应的虚拟设备实例，建立通信连接。

设备挂载的接口定义如下：

```

1 def attach_device(self, address, device):
2     """
3     将虚拟设备挂载到 I2C 总线
4     参数：
5         address: 7 位从机地址
6         device: 虚拟设备实例，需实现 read/write 方法
7     """
8     self.devices[address] = device
    
```

Listing 2: 虚拟 I2C 设备挂载接口

(2) **读写操作**：虚拟 I2C 支持字节级和缓冲区级的读写操作。写操作时，主设备发送从机地址和写标志位 (0)，随后发送要写入的寄存器地址和数据字节；读操作时，主设备先发送从机地址和写标志位以指定寄存器地址，然后重新发送起始信号和从机地址与读标志位 (1)，从设备返回请求的数据字节。

(3) **错误处理**：虚拟 I2C 实现了基本的错误检测机制，包括：地址无应答 (NACK)——当发送的从机地址在设备列表中找不到匹配项时返回错误；总线忙——当多个主设备同时尝试控制总线时产生冲突检测；数据校验——支持对传输数据进行 CRC 校验 (可选)。

3.3.4 UART 串口通信

UART(Universal Asynchronous Receiver/Transmitter)是一种异步串行通信协议,广泛用于微控制器与 PC、蓝牙模块、GPS 模块等设备之间的数据传输。STM32F103C8T6 提供三个独立的 USART 接口，支持全双工通信和多种波特率配置。

虚拟 UART 模块的设计要点包括：

(1) **波特率配置**：波特率表示每秒传输的比特数，是 UART 通信的关键参数。虚拟 UART 记录当前配置的波特率值，虽然由于软件模拟的本质无法真正按照波特率进行位级时序控制，但波特率值会影响某些时序相关的模拟行为（如超时设置）。常见的波特率值包括 9600、115200、921600 等。

(2) **printf 重定向模拟**：在嵌入式开发中，通常将标准输出函数 printf 重定向到 UART，以便通过串口输出调试信息。虚拟 UART 提供了等效的字符串发送接口，将待发送的字符串缓存到内部缓冲区中，并触发接收端回调函数。

(3) **回调机制**：虚拟 UART 支持注册接收回调函数，当有数据到达时自动调用。这一机制模拟了真实 UART 的接收中断，使得上层代码能够以事件驱动的方式处理串口数据，而不需要轮询接收缓冲区。

```

1 class VirtualUART:
2     def __init__(self):
3         self.baudrate = 115200
4         self.rx_callback = None
5         self.tx_buffer = []
6
7     def set_rx_callback(self, callback):
8         """注册接收回调函数"""
9         self.rx_callback = callback
10
11    def send(self, data):
12        """发送数据（字符串或字节）"""
13        if isinstance(data, str):
14            data = data.encode('utf-8')
15        self.tx_buffer.extend(data)
16        if self.rx_callback:
17            self.rx_callback(data)
18
19    def printf(self, fmt, *args):
20        """格式化输出，模拟printf"""
21        msg = fmt % args
22        self.send(msg)

```

Listing 3: 虚拟 UART 回调注册与发送

如代码 3 所示，VirtualUART 类提供了简洁的接口用于数据发送和回调注册。send 方法接受字符串或字节类型的数据，printf 方法则模拟了 C 语言 printf 函数的格式化输出功能。

3.3.5 TIM 通用定时器

TIM (Timer) 是 STM32 中功能最丰富的外设之一，可用作定时器、计数器、PWM 发生器和输入捕获器等。本系统主要使用 TIM 的基本定时功能，为传感器数据采集提供周期性的触发信号。

虚拟 TIM 模块的实现基于 Python 的 threading 模块，采用守护线程 (Daemon Thread) 在后台运行。守护线程的特点是当主线程结束时，守护线程会自动终止，无需显式关闭，适合用于周期性的后台任务。

虚拟 TIM 的核心工作流程如下：

(1) **计数器递增**：TIM 启动后，守护线程以固定的时间间隔（由预分频系数和自动

重载值决定) 递增内部计数器。计数器的递增频率计算公式为:

$$f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)} \quad (4)$$

其中, f_{clk} 为定时器时钟频率 (通常为 72 MHz), PSC 为预分频器值, ARR 为自动重载值。

(2) **周期回调**: 当计数器达到自动重载值时, 计数器归零并触发更新事件 (Update Event)。如果已注册周期回调函数, 则在此时调用该函数。在环境监测系统中, 周期回调函数通常执行传感器数据采集任务, 从而实现固定频率的自动采样。

(3) **启动与停止**: TIM 通过 CR1 寄存器的 CEN 位控制启动和停止。将 CEN 位置 1 启动定时器, 清零则停止定时器。虚拟 TIM 在启动时创建并启动守护线程, 在停止时设置一个事件标志通知线程退出。

```

1 import threading
2 import time
3
4 class VirtualTIM:
5     def __init__(self):
6         self.prescaler = 7199          # PSC
7         self.auto_reload = 9999        # ARR
8         self.counter = 0
9         self.running = False
10        self.period_callback = None
11        self._stop_event = threading.Event()
12
13    def set_period_callback(self, callback):
14        """设置周期回调函数"""
15        self.period_callback = callback
16
17    def _timer_thread(self):
18        """定时器守护线程"""
19        period_ms = (self.prescaler + 1) * (self.auto_reload + 1) /
20            72000
21        while not self._stop_event.is_set():
22            time.sleep(period_ms / 1000.0)
23            self.counter = 0
24            if self.period_callback:
25                self.period_callback()
26
27    def start(self):
28        """启动定时器"""
29        self.running = True
    
```

```

29         self._stop_event.clear()
30         t = threading.Thread(target=self._timer_thread, daemon=True)
31         t.start()
32
33     def stop(self):
34         """停止定时器"""
35         self.running = False
36         self._stop_event.set()

```

Listing 4: 虚拟 TIM 守护线程实现

如代码 4 所示, VirtualTIM 类通过守护线程实现了周期性的定时功能。period_callback 是用户注册的回调函数, 在每个定时周期结束时被调用。通过调整 prescaler 和 auto_reload 的值, 可以灵活配置定时周期, 适应不同的采样频率需求。

4 传感器驱动与数据采集

传感器是环境监测系统的“器官”, 负责将物理世界中的环境参数转换为电信号, 再由微控制器进行处理和分析。在虚拟化平台中, 传感器的模拟是连接虚拟硬件与上层应用的关键桥梁, 其模拟精度直接决定了整个系统的可信度。本章首先介绍传感器虚拟化的通用建模框架, 然后详细阐述各传感器的模拟原理与驱动实现。

4.1 传感器虚拟化原理

4.1.1 通用建模框架

为了实现传感器模拟的一致性和可扩展性, 本系统定义了一套通用的传感器建模框架。该框架将传感器抽象为三个核心组成部分: 环境参数输入、物理量转换模型和数字量输出。其工作流程如图 ?? 所示 (概念描述)。

通用建模框架的核心要素包括:

(1) **环境参数定义**: 每个传感器关联一组环境参数变量, 这些变量代表了传感器所测量的物理量的真实值。例如, 温度传感器关联 temperature 参数, 光照传感器关联 illuminance 参数。环境参数的值可以通过控制面板实时调整, 模拟不同的环境条件。

(2) **物理量转换模型**: 描述环境参数如何被转换为传感器输出的物理量 (通常是电压、电阻或数字量)。不同传感器的转换模型各不相同, 可能是线性关系、幂律关系或分段函数关系。转换模型应基于传感器的数据手册 (Datasheet) 和物理原理建立, 确保模拟数据的合理性。

(3) **噪声模型**: 真实传感器的输出不可避免地包含随机噪声, 噪声的来源包括热噪声、散粒噪声和电磁干扰等。为了增强模拟的真实性, 在传感器的理想输出上叠加高斯

白噪声。噪声模型的数学表达式为：

$$Y = X + N(0, \sigma^2) \quad (5)$$

其中， Y 为传感器的实际输出值， X 为理想输出值（由物理量转换模型计算得到）， $N(0, \sigma^2)$ 为服从均值为 0、方差为 σ^2 的正态分布的随机噪声。

噪声强度 σ 的取值依据传感器的精度等级和典型应用场景确定。对于高精度传感器（如 BMP280）， σ 取较小的值；对于低精度传感器（如 MQ-135）， σ 取较大的值。在特殊场景下，还可以通过增加异常值（Outlier）模拟传感器故障或环境突变。

4.1.2 传感器驱动接口

所有虚拟传感器驱动均遵循统一的接口规范，便于上层代码以一致的方式访问不同类型的传感器。核心接口包括：

- `init()`：初始化传感器，配置通信参数和工作模式。
- `read()`：读取传感器的当前测量值，返回原始数据或经过校准的物理量。
- `calibrate()`：执行校准操作，更新校准系数（如适用）。
- `set_environment(env_dict)`：更新传感器关联的环境参数。

统一的接口设计使得新增传感器类型时，只需实现上述接口即可无缝集成到现有系统中，无需修改数据采集循环和 Web 展示层的代码。

4.2 DHT11 温湿度传感器

4.2.1 传感器概述

DHT11 是一款集成了温度测量和湿度测量功能的数字传感器，采用单总线（One-Wire）串行通信协议。该传感器价格低廉、体积小巧、使用简单，是入门级物联网项目中最常用的温湿度传感器之一。其主要技术参数如表 4 所示。

表 4: DHT11 主要技术参数

参数项	规格
测量范围（湿度）	20% – 90% RH
测量精度（湿度）	±5% RH
测量范围（温度）	0°C – 50°C
测量精度（温度）	±2°C
分辨率	1% RH / 1°C
采样周期	≥ 1 秒
供电电压	3.3 V – 5.5 V
通信协议	单总线串行

4.2.2 单总线协议模拟

DHT11 的单总线通信协议分为三个阶段：主机启动信号、DHT11 响应信号和数据传输。

(1) **主机启动信号**：主机 (MCU) 将数据线拉低至少 18 毫秒，然后释放数据线，等待 DHT11 响应。这一阶段的目的是唤醒处于低功耗状态的 DHT11 传感器，通知其准备发送数据。

(2) **DHT11 响应信号**：DHT11 检测到启动信号后，先将数据线拉低 80 微秒，再拉高 80 微秒，表示已准备好发送数据。主机通过检测这一响应信号确认 DHT11 在线且工作正常。

(3) **数据传输**：DHT11 以 40 位数据帧的格式发送温湿度信息。每一位数据以 50 微秒的低电平起始，随后高电平的持续时间决定该位的值：高电平持续 26–28 微秒表示 0，持续 70 微秒表示 1。40 位数据包括：8 位湿度整数、8 位湿度小数、8 位温度整数、8 位温度小数和 8 位校验和。

虚拟 DHT11 驱动实现了上述协议的完整模拟，核心代码如下：

```

1 class VirtualDHT11:
2     def __init__(self, gpio, pin):
3         self.gpio = gpio
4         self.pin = pin
5         self.humidity = 50.0      # 默认湿度 50%
6         self.temperature = 25.0   # 默认温度 25C
7
8     def set_environment(self, env):
9         self.humidity = env.get('humidity', 50.0)
10        self.temperature = env.get('temperature', 25.0)
11
12    def read(self):
13        # 模拟启动信号
14        self.gpio.set_mode(self.pin, 'OUTPUT')
15        self.gpio.write(self.pin, 0)
16        delay_ms(20)
17        self.gpio.write(self.pin, 1)
18        delay_us(30)
19        self.gpio.set_mode(self.pin, 'INPUT')
20
21        # 模拟响应信号和数据传输
22        # 返回 (湿度, 温度)
23        h_int = int(self.humidity)
24        h_dec = int((self.humidity - h_int) * 10)
25        t_int = int(self.temperature)
26        t_dec = int((self.temperature - t_int) * 10)

```

```

27         checksum = (h_int + h_dec + t_int + t_dec) & 0xFF
28
29         data = [h_int, h_dec, t_int, t_dec, checksum]
30         return self.humidity + random.gauss(0, 1.5), \
31                self.temperature + random.gauss(0, 0.8)

```

Listing 5: DHT11 启动信号与数据读取

如代码 5 所示, 虚拟 DHT11 驱动通过控制 GPIO 引脚电平模拟单总线协议的关键时序。在数据生成阶段, 将环境参数转换为 40 位数据格式, 并计算校验和。同时, 按照公式 5 叠加高斯噪声, 湿度噪声标准差取 1.5% RH, 温度噪声标准差取 0.8°C, 与 DHT11 的实际精度等级相匹配。

4.3 MQ-135 空气质量传感器

4.3.1 传感器原理

MQ-135 是一种广谱型气体传感器, 对氨气、硫化物、苯系蒸汽、烟雾和有害气体等多种气体具有较高的灵敏度。其敏感元件为二氧化锡 (SnO_2) 半导体材料, 当环境中存在目标气体时, 气体分子在敏感材料表面发生氧化还原反应, 导致材料的电阻值发生变化。通过测量电阻变化量, 可以间接推算出气体浓度。

MQ-135 的敏感元件电阻 R_S 与气体浓度 C 之间存在幂律关系:

$$\frac{R_S}{R_0} = A \cdot C^{-B} \quad (6)$$

其中, R_0 为传感器在清洁空气中的基准电阻值, A 和 B 为与气体类型相关的经验常数, C 为气体浓度 (单位为 ppm)。对于一般空气质量监测场景, 通常取 $A \approx 116.6$, $B \approx 2.77$ 。

4.3.2 电压-浓度反推

MQ-135 传感器模块通常采用简单的分压电路将电阻变化转换为电压输出。分压电路的输出电压 V_{out} 与敏感元件电阻 R_S 的关系为:

$$V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L} \quad (7)$$

其中, V_{cc} 为供电电压 (通常为 5 V), R_L 为负载电阻 (可调电位器)。

由此可以反推敏感元件电阻:

$$R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \quad (8)$$

进一步代入公式 6，可得气体浓度：

$$C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}} \quad (9)$$

4.3.3 虚拟驱动实现

虚拟 MQ-135 驱动的核心任务是根据环境参数中的 air_quality 值（单位为 ppm），按照公式 6和 9计算对应的 ADC 电压值。为了模拟真实传感器的行为，还实现了以下特性：

(1) **预热模拟**：MQ-135 传感器在通电后需要一段时间预热才能达到稳定工作状态，典型预热时间为 20 秒。虚拟驱动在初始化后记录启动时间，在预热期间返回无效数据或较低精度的数据。

(2) **非线性响应**：按照幂律关系，气体浓度与电阻值之间呈非线性关系。在低浓度区域，传感器的灵敏度较高；在高浓度区域，灵敏度逐渐降低。虚拟驱动完整模拟了这一非线性特性。

(3) **温湿度补偿**：MQ-135 的输出受环境温湿度影响。虚拟驱动引入了简单的温湿度补偿系数，根据当前温度和湿度对浓度值进行修正，提高模拟数据的真实性。

4.4 BH1750 光照传感器

4.4.1 传感器概述

BH1750 是一款数字型光照强度传感器，采用 I2C 通信接口，可直接输出以勒克斯 (lux) 为单位的光照度数值，无需进行复杂的模数转换和校准计算。该传感器量程广、精度高、使用简便，广泛应用于自动照明控制、背光调节和环境监测等领域。

BH1750 支持多种测量模式，通过向传感器发送不同的命令字节来切换。常用命令如表 5所示。

表 5: BH1750 常用 I2C 命令

命令值	名称	功能描述
0x01	Power On	上电命令，唤醒传感器
0x00	Power Down	掉电命令，进入低功耗模式
0x10	Continuous H-Res Mode	连续高分辨率模式（1 lux 分辨率，120ms 测量时间）
0x11	Continuous H-Res2 Mode	连续高分辨率模式 2（0.5 lux 分辨率，120ms 测量时间）
0x13	Continuous L-Res Mode	连续低分辨率模式（4 lux 分辨率，16ms 测量时间）
0x20	One Time H-Res Mode	单次高分辨率模式

4.4.2 光照计算公式

BH1750 在高分辨率模式 (H-Res Mode) 下的输出值为 16 位原始数据 (raw value), 需要乘以灵敏度系数才能得到以 lux 为单位的光照度。标准灵敏度系数为 1.2 lux/计数, 计算公式为:

$$E_v = \text{raw} \times 1.2 \quad (10)$$

其中, E_v 为光照度 (单位: lux), raw 为传感器返回的 16 位原始数据。

例如, 当原始数据为 500 时, 对应的光照度为 $500 \times 1.2 = 600 \text{ lux}$ 。

虚拟 BH1750 驱动通过 I2C 接口与 MCU 通信, 接收命令字节后根据当前环境参数中的 illuminance 值, 按照公式 10 反推原始数据:

$$\text{raw} = \left\lfloor \frac{E_v}{1.2} \right\rfloor + N(0, \sigma^2) \quad (11)$$

其中, σ 取 3 个计数单位, 对应约 3.6 lux 的测量噪声, 与 BH1750 的数据手册规格相符。

4.5 BMP280 气压传感器

4.5.1 传感器概述

BMP280 是博世公司推出的一款高精度气压和温度传感器, 采用 MEMS 技术制造, 具有体积小、功耗低、精度高等优点。该传感器广泛应用于气象监测、海拔测量、室内导航和无人机高度控制等领域。其主要技术参数如表 6 所示。

表 6: BMP280 主要技术参数

参数项	规格
气压测量范围	300 hPa – 1100 hPa
气压测量精度	$\pm 0.12 \text{ hPa}$ (典型值)
温度测量范围	$-40^\circ\text{C} - +85^\circ\text{C}$
温度测量精度	$\pm 0.5^\circ\text{C}$ (25°C 时)
绝对精度	$\pm 1 \text{ hPa}$
通信接口	I2C 或 SPI
芯片 ID	0x58

4.5.2 芯片 ID 验证

BMP280 内部有一个只读的芯片 ID 寄存器, 地址为 0xD0, 上电后的默认值为 0x58。读取该寄存器是验证传感器是否正确连接和工作的常用方法。虚拟 BMP280 驱动在初始化时模拟了这一行为, 当主机读取 0xD0 寄存器时返回 0x58。

4.5.3 24 位 ADC 与校准系数

BMP280 内部集成了 24 位 ADC，用于将气压和温度传感器元件输出的模拟信号转换为数字量。气压 ADC 输出为 20 位有效数据（存储在 3 个字节中），温度 ADC 输出也为 20 位有效数据。高分辨率确保了传感器能够分辨极微小的气压变化。

真实 BMP280 在出厂时烧录了一组校准系数（Calibration Coefficients），存储在 0x88-0xA1 地址范围的 NVM 中。主机在读取测量数据前，需要先读取这些校准系数，然后在软件中执行补偿算法，将原始 ADC 值转换为补偿后的物理量。补偿算法的公式较为复杂，涉及二次和三次多项式计算。

在虚拟平台中，为了简化实现同时保持数据的合理性，采用了简化的校准模型。原始 ADC 值与环境参数之间的映射关系使用线性公式近似：

$$\text{raw_P} = \left\lfloor \frac{P}{1013.25} \times 415148 \right\rfloor + N(0, \sigma_P^2) \quad (12)$$

$$\text{raw_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2) \quad (13)$$

其中， P 为环境气压值（单位：hPa）， T 为环境温度值（单位：°C），415148 是 1013.25 hPa（标准大气压）对应的典型原始 ADC 值， σ_P 和 σ_T 为噪声标准差。

公式 12 的推导依据为：BMP280 的原始 ADC 值在标准大气压附近约为 415148（对应过采样率 *imes16* 模式），因此建立了气压值与原始值之间的比例关系。虽然简化了真实 BMP280 复杂的温度补偿和非线性校准过程，但对于教学演示和一般性验证已足够精确。

4.6 噪音传感器

4.6.1 传感器原理

噪音传感器（声音传感器）用于测量环境噪声的强度，通常以分贝（dB）为单位表示。常见的噪音传感器模块（如 KY-038 或 LM393-based 模块）集成了高灵敏度麦克风和信号调理电路，将声音信号转换为模拟电压输出。

分贝是一种对数单位，用于表示两个功率值或声压值之比。声压级（Sound Pressure Level, SPL）的定义公式为：

$$L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right) \quad (14)$$

其中， L_p 为声压级（单位：dB）， p 为实际声压（单位：Pa）， p_0 为参考声压（通常为 20×10^{-6} Pa，即人耳在 1 kHz 频率下的听阈）。

4.6.2 线性电压映射

噪音传感器模块通常输出与环境声压成正比的模拟电压信号（经过放大和整流处理）。为了简化虚拟模型，假设传感器输出电压与声压级之间存在线性映射关系：

$$V_{out} = V_{offset} + k \cdot L_p \quad (15)$$

其中, V_{offset} 为偏移电压 (对应 0 dB 时的输出, 实际中通常为一个较小的正值), k 为灵敏度系数 (单位: V/dB)。

在虚拟驱动中, 根据环境参数中的 `noise_level` 值 (单位: dB), 按照公式 15 计算输出电压, 再通过 ADC 转换为数字量。为保证模拟的真实性, 在最终输出上叠加适当的高斯噪声, 标准差取 1.5 dB, 对应一般消费级噪音传感器的精度水平。

虚拟噪音传感器还模拟了以下特性:

(1) **方向性**: 真实麦克风具有一定的指向性, 对来自不同方向的声音灵敏度不同。虚拟传感器引入了简单的方向性系数, 模拟这一物理特性。

(2) **频率响应**: 人耳对不同频率声音的敏感度不同, A 计权 (A-weighting) 是最常用的声压级计权方式。虚拟传感器默认输出 A 计权声压级, 更接近人耳的主观感受。

(3) **峰值保持**: 部分噪音传感器支持峰值检测功能, 记录一段时间内的最大声压级。虚拟传感器实现了可选的峰值保持模式, 在指定时间窗口内记录并返回最大 dB 值。

4.7 数据采集循环设计

数据采集循环是整个虚拟环境监测系统的核心业务流程, 负责以固定的周期驱动各传感器进行测量, 并将采集到的数据送入存储和展示模块。一个完整的数据采集周期包括以下 6 个步骤:

1. **读取环境参数**: 从共享文件 `env_control.txt` 中读取当前的环境参数设置, 包括温度、湿度、气压、光照、空气质量和噪音等级。
2. **更新传感器环境**: 将读取到的环境参数传递给各虚拟传感器实例, 使传感器能够根据最新的环境设置生成模拟数据。
3. **执行传感器读取**: 依次调用各传感器的 `read()` 方法, 获取当前的测量值。读取顺序通常为: DHT11 (温湿度) → BMP280 (气压/温度) → BH1750 (光照) → MQ-135 (空气质量) → 噪音传感器。
4. **数据格式化**: 将各传感器的原始读数转换为标准单位 (如 lux、hPa、ppm、dB 等), 并按照预定义的数据结构组织为字典或 JSON 格式。
5. **数据存储**: 将格式化后的数据记录插入到 SQLite 数据库的 `sensor_data` 表中, 同时更新内存中的最新数据缓存。
6. **告警检查**: 遍历各传感器的告警配置, 检查当前读数是否超出阈值范围。如有超限, 则记录告警日志并更新告警状态。

为了保证数据采集周期的稳定性, 避免因单次采集耗时过长而导致周期抖动, 引入了采样周期补偿机制。设目标采样周期为 T_{sample} (单位: 秒), 单次采集的实际耗时为 $t_{elapsed}$, 则本次采集后的休眠时间应为:

$$t_{sleep} = \max(0, T_{sample} - t_{elapsed}) \quad (16)$$

公式 16 确保了整个数据采集循环的实际周期不会小于目标周期。当单次采集耗时超过目标周期时，循环将立即进入下一轮，不做额外等待，以保证数据的实时性优先。

```

1 import time
2
3 SAMPLE_PERIOD = 2.0  # 采样周期 2 秒
4
5 def data_collection_loop(sensors, db, alarm_config):
6     """数据采集主循环"""
7     while True:
8         t_start = time.time()
9
10        # Step 1: 读取环境参数
11        env = read_environment_control()
12
13        # Step 2: 更新传感器环境
14        for sensor in sensors:
15            sensor.set_environment(env)
16
17        # Step 3: 执行传感器读取
18        readings = {}
19        for sensor in sensors:
20            readings.update(sensor.read())
21
22        # Step 4: 数据格式化
23        record = format_sensor_data(readings)
24
25        # Step 5: 数据存储
26        db.insert_sensor_data(record)
27
28        # Step 6: 告警检查
29        check_alarms(record, alarm_config, db)
30
31        # 周期补偿
32        t_elapsed = time.time() - t_start
33        t_sleep = max(0, SAMPLE_PERIOD - t_elapsed)
34        time.sleep(t_sleep)

```

Listing 6: 数据采集循环核心实现

如代码 6 所示，data_collection_loop 函数实现了完整的 6 步数据采集流程。SAMPLE_PERIOD 定义了目标采样周期，通过周期补偿机制确保循环周期的稳定性。各步

骤之间通过 readings 字典传递数据，最终格式化的 record 被存入数据库并用于告警检查。

5 软件系统设计与实现

在虚拟 STM32 硬件平台和传感器驱动的基础上，本章将介绍上层软件系统的设计与实现，包括数据存储模块、告警系统、Web 可视化系统和控制面板四个核心组成部分。这些模块共同构成了完整的智能环境监测站应用，实现了数据的持久化管理、异常事件的自动检测、监测结果的直观展示以及环境参数的远程调控功能。

5.1 数据存储模块

数据存储是任何监测系统的核心功能之一，负责将采集到的传感器数据、系统运行状态和告警事件等信息持久化保存，以支持历史查询、统计分析和审计追溯等后续操作。本系统采用 SQLite 作为数据库引擎，具有零配置、单文件存储和易于部署等优点。

5.1.1 数据库表结构设计

数据库共包含三张核心表：sensor_data（传感器数据表）、alarm_log（告警记录表）和 system_log（系统日志表）。

(1) **sensor_data 表**：用于存储各传感器的实时采集数据，是数据量最大的表。其结构设计如表 7 所示，共包含 10 个字段。

表 7: sensor_data 表结构

字段名	数据类型	约束	说明
id	INTEGER	PRIMARY KEY AUTOINCREMENT	自增主键
timestamp	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	采集时间戳
temperature	REAL		温度值，单位 °C
humidity	REAL		湿度值，单位% RH
pressure	REAL		气压值，单位 hPa
illuminance	REAL		光照度，单位 lux
air_quality	REAL		空气质量指数，单位 ppm
noise_level	REAL		噪音等级，单位 dB
cpu_temp	REAL		MCU 内部温度，单位 °C
data_source	TEXT	DEFAULT 'simulation'	数据来源标识

id 字段为自增主键，确保每条记录具有唯一标识；timestamp 字段自动记录数据插入时间，便于按时间范围查询；temperature 至 noise_level 六个字段分别对应六种环境参数的测量值；cpu_temp 字段记录虚拟 MCU 的内部温度（由内部温度传感器模拟）；data_source 字段标识数据的来源，默认值为'simulation'，便于后续扩展支持真实硬件数据接入。

(2) **alarm_log 表**：用于记录告警事件的发生、持续和恢复过程。字段包括：id(主键)、timestamp(告警时间)、sensor_type(传感器类型)、alarm_type(告警类型：HIGH/LOW)、threshold(阈值)、actual_value(实际值)、status(状态：ACTIVE/RESOLVED)和 resolved_at(恢复时间)。

(3) **system_log 表**：用于记录系统运行过程中的关键事件和调试信息。字段包括：id(主键)、timestamp(日志时间)、level(日志级别：DEBUG/INFO/WARNING/ERROR)、module(模块名称)、message(日志内容)和 metadata(附加元数据，JSON 格式)。

5.1.2 Python sqlite3 接口

Python 标准库中的 sqlite3 模块提供了对 SQLite 数据库的完整访问接口。通过该模块，可以执行 SQL 语句、管理事务、获取查询结果，而无需安装额外的数据库驱动程序。

数据库连接采用上下文管理器 (Context Manager) 模式，确保每次数据库操作后连接能够正确关闭，避免资源泄漏。同时，通过设置 journal_mode 为 WAL (Write-Ahead Logging)，提高了并发读写性能，使得数据采集线程和 Web 服务线程能够同时访问数据库而不会产生严重的锁竞争。

5.1.3 CRUD 操作

数据存储模块封装了完整的 CRUD (Create, Read, Update, Delete) 操作接口：

(1) **Create (创建)**：insert_sensor_data 方法接收一个包含各传感器读数的字典，将其插入到 sensor_data 表中。插入操作使用参数化查询 (Parameterized Query)，有效防止 SQL 注入攻击。

(2) **Read (读取)**：提供了多种查询方法，包括：get_latest_reading() 获取最新的一条记录；get_readings_by_range(start,end) 获取指定时间范围内的记录；get_readings_by_sensor(sensor_type,limit) 获取某传感器的最近 N 条记录。

(3) **Update (更新)**：update_alarm_status 方法用于更新告警记录的状态（从 ACTIVE 更新为 RESOLVED），并设置 resolved_at 字段为当前时间。

(4) **Delete (删除)**：delete_old_readings(before_date) 方法用于删除指定日期之前的历史数据，支持定时清理和手动清理两种模式，防止数据库文件无限增长。

5.1.4 统计查询

除了基本的增删改查操作外，数据存储模块还提供了丰富的统计查询功能，支持对历史数据进行聚合分析。常用的统计查询包括：

(1) **平均值查询 (AVG)**：计算指定时间段内某传感器的平均读数，反映该时段内的

总体水平。

(2) **最小值查询 (MIN)**: 计算指定时间段内某传感器的最小读数, 用于识别极端低值事件。

(3) **最大值查询 (MAX)**: 计算指定时间段内某传感器的最大读数, 用于识别极端高值事件。

(4) **标准差查询 (STDEV)**: 计算指定时间段内某传感器读数的标准差, 反映数据的波动程度。

统计查询的 SQL 模板如下:

```

1 SELECT
2     AVG(temperature) as avg_temp,
3     MIN(temperature) as min_temp,
4     MAX(temperature) as max_temp,
5     AVG(humidity) as avg_humidity,
6     MIN(pressure) as min_pressure,
7     MAX(pressure) as max_pressure
8 FROM sensor_data
9 WHERE timestamp BETWEEN ? AND ?;
```

Listing 7: 统计查询 SQL 模板

如代码 7 所示, 通过 SQL 聚合函数可以一次性获取多个传感器的统计信息。参数化的时间范围条件保证了查询的灵活性和安全性。

5.2 告警系统设计

告警系统是智能环境监测站的重要组成部分, 负责实时监测各传感器的读数, 当检测到异常值时及时发出告警通知, 帮助用户快速响应环境变化。

5.2.1 告警阈值配置

告警阈值通过 ALARM_CONFIG 配置字典定义, 支持为每个传感器设置独立的高阈值 (high_threshold) 和低阈值 (low_threshold)。当传感器读数超过高阈值时触发高温/高浓度类告警, 低于低阈值时触发低温/低浓度类告警。

默认的告警配置示例如下:

```

1 ALARM_CONFIG = {
2     'temperature': {
3         'high_threshold': 35.0,    # 高温告警阈值 35C
4         'low_threshold': 10.0,    # 低温告警阈值 10C
5         'unit': 'C',
6         'name': '温度'
7     },
```

```

8      'humidity': {
9          'high_threshold': 80.0,    # 高湿告警阈值 80%
10         'low_threshold': 20.0,    # 低湿告警阈值 20%
11         'unit': '% RH',
12         'name': '湿度'
13     },
14     'pressure': {
15         'high_threshold': 1050.0, # 高气压告警阈值 1050 hPa
16         'low_threshold': 980.0,   # 低气压告警阈值 980 hPa
17         'unit': 'hPa',
18         'name': '气压'
19     },
20     'air_quality': {
21         'high_threshold': 500.0,   # 空气质量告警阈值 500 ppm
22         'low_threshold': None,     # 无低阈值
23         'unit': 'ppm',
24         'name': '空气质量'
25     },
26     # ... 其他传感器配置
27 }

```

Listing 8: 告警阈值配置示例

如代码 8 所示，每个传感器的配置项包括阈值数值、单位、中文名称等。当某传感器的 `low_threshold` 设为 `None` 时，表示不监测低值告警，只监测高值告警。

5.2.2 告警检查逻辑

`check_all` 函数是告警系统的核心，负责遍历所有已配置的传感器，检查当前读数是否超出阈值范围。其检查逻辑如下：

(1) **遍历检查**：对于 `ALARM_CONFIG` 中的每个传感器配置项，获取当前读数和对应的阈值设置。

(2) **高阈值判断**：如果当前读数大于 `high_threshold`，则触发 `HIGH` 类型告警。如果该传感器此前已存在 `ACTIVE` 状态的 `HIGH` 告警，则更新其实际值；否则创建新的告警记录。

(3) **低阈值判断**：如果当前读数小于 `low_threshold`，则触发 `LOW` 类型告警。判断逻辑与高阈值类似。

(4) **告警恢复**：如果当前读数已回到正常范围内，且此前存在 `ACTIVE` 状态的告警，则将该告警标记为 `RESOLVED` 状态，并记录恢复时间。

(5) **持久化**：新创建的告警记录和状态变更均通过数据库接口持久化保存，确保告警历史可追溯。

```

1 def check_all(readings, config, db):
2     """
3     检查所有传感器的告警状态
4     参数:
5         readings: 当前传感器读数字典
6         config:   告警配置字典
7         db:       数据库操作实例
8     """
9     for sensor, cfg in config.items():
10        value = readings.get(sensor)
11        if value is None:
12            continue
13
14        # 高阈值检查
15        high = cfg.get('high_threshold')
16        if high is not None and value > high:
17            trigger_alarm(db, sensor, 'HIGH', high, value)
18
19        # 低阈值检查
20        low = cfg.get('low_threshold')
21        if low is not None and value < low:
22            trigger_alarm(db, sensor, 'LOW', low, value)
23
24        # 恢复检查
25        if (high is None or value <= high) and \
26            (low is None or value >= low):
27            resolve_alarm(db, sensor)

```

Listing 9: 告警检查核心逻辑

如代码 9 所示, check_all 函数实现了完整的告警检查流程。trigger_alarm 和 resolve_alarm 为辅助函数, 分别负责创建新告警和恢复已有告警。

5.2.3 告警记录持久化

每条告警记录包含完整的上下文信息, 便于事后分析和审计。告警记录的数据模型如表 8 所示。

表 8: alarm_log 表结构

字段名	数据类型	约束	说明
id	INTEGER	PRIMARY KEY AUTOINCREMENT	自增主键
timestamp	DATETIME	NOT NULL	告警触发时间
sensor_type	TEXT	NOT NULL	传感器类型
alarm_type	TEXT	NOT NULL	告警类型: HIGH/LOW
threshold	REAL	NOT NULL	触发阈值
actual_value	REAL	NOT NULL	触发时的实际值
status	TEXT	DEFAULT 'ACTIVE'	状态: ACTIVE/RESOLVED
resolved_at	DATETIME		恢复时间

5.3 Web 可视化系统

Web 可视化系统是用户与虚拟环境监测站交互的主要界面，负责将底层采集到的传感器数据以直观、美观的形式呈现出来，并提供环境参数控制和告警信息查看等功能。

5.3.1 Flask 路由设计

后端采用 Flask 框架提供 Web 服务，定义了多个 RESTful API 端点供前端调用。各端点的功能如表 9所示。

表 9: Flask RESTful API 路由设计

方法	路径	功能	说明
GET	/	主页	返回 Web 可视化页面
GET	/api/sensors/latest	最新数据	返回所有传感器的最新读数
GET	/api/sensors/history	历史数据	返回指定时间范围内的历史数据
GET	/api/alarms/active	活动告警	返回当前未恢复的告警列表
GET	/api/alarms/history	告警历史	返回所有告警记录（支持分页）
POST	/api/control	环境控制	接收环境参数设置指令
GET	/api/stats	统计数据	返回指定时段的统计信息

5.3.2 RESTful JSON 响应格式

所有 API 端点均采用统一的 JSON 响应格式，包含 status、data 和 message 三个顶层字段。status 字段为'ok' 或'error'，表示请求处理状态；data 字段包含实际的响应数据；message 字段在出错时提供错误描述信息。

成功响应示例：

```
1 {
2   "status": "ok",
```

```

3      "data": {
4          "temperature": 26.5,
5          "humidity": 58.2,
6          "pressure": 1013.8,
7          "illuminance": 320.0,
8          "air_quality": 120.5,
9          "noise_level": 45.3,
10         "timestamp": "2024-01-15T09:30:00"
11     },
12     "message": null
13 }

```

Listing 10: API 成功响应示例

5.3.3 Chart.js 双折线图配置

前端采用 Chart.js 库绘制传感器数据的实时趋势图。为了在同一图表中展示两种相关传感器的对比趋势，系统使用了双折线图（Dual Line Chart）配置。以温湿度趋势图为例，其 Chart.js 配置如下：

```

1  const ctx = document.getElementById('tempHumChart').getContext('2d');
2  const tempHumChart = new Chart(ctx, {
3      type: 'line',
4      data: {
5          labels: [],          // 时间标签数组
6          datasets: [
7              {
8                  label: '温度 (C)',
9                  data: [],
10                 borderColor: 'rgb(255, 99, 132)',
11                 backgroundColor: 'rgba(255, 99, 132, 0.1)',
12                 yAxisID: 'y-temp',
13                 tension: 0.3
14             },
15             {
16                 label: '湿度 (% RH)',
17                 data: [],
18                 borderColor: 'rgb(54, 162, 235)',
19                 backgroundColor: 'rgba(54, 162, 235, 0.1)',
20                 yAxisID: 'y-humidity',
21                 tension: 0.3
22             }
16

```

```

23     ]
24   },
25   options: {
26     responsive: true,
27     interaction: { mode: 'index', intersect: false },
28     scales: {
29       'y-temp': {
30         type: 'linear',
31         display: true,
32         position: 'left',
33         title: { display: true, text: '温度 (C)' }
34       },
35       'y-humidity': {
36         type: 'linear',
37         display: true,
38         position: 'right',
39         title: { display: true, text: '湿度 (% RH)' }
40       }
41     }
42   }
43 });

```

Listing 11: Chart.js 双折线图配置

如代码 11 所示，双折线图通过 `yAxisID` 属性将两条曲线分别绑定到左右两个 Y 轴上，避免了因量纲不同而导致的数据可视化失真。`tension` 属性控制曲线的平滑程度，设置为 0.3 可在保持趋势清晰的同时增加一定的视觉平滑效果。

5.3.4 1 秒轮询机制

为了实现数据的实时更新，前端采用 1 秒间隔的定时轮询（Polling）机制，定期向后端 API 请求最新的传感器数据和告警状态。轮询机制的实现基于 JavaScript 的 `setInterval` 函数：

```

1 // 每1000毫秒（1秒）轮询一次
2 setInterval(() => {
3   fetch('/api/sensors/latest')
4     .then(r => r.json())
5     .then(data => {
6       if (data.status === 'ok') {
7         updateSensorCards(data.data);
8         updateCharts(data.data);
9       }

```

```

10     })
11     .catch(err => console.error('Polling error:', err));
12
13     fetch('/api/alarms/active')
14     .then(r => r.json())
15     .then(data => {
16         if (data.status === 'ok') {
17             updateAlarmPanel(data.data);
18         }
19     });
20 }, 1000);

```

Listing 12: 前端轮询机制实现

如代码 12 所示，前端同时轮询最新传感器数据和活动告警信息，确保界面上显示的数据与后端状态保持同步。虽然轮询机制相比 WebSocket 等推送技术的实时性稍差，但其实现简单、兼容性好，对于 1 秒更新频率的应用场景已足够使用。未来可考虑引入 Server-Sent Events (SSE) 或 WebSocket 以降低网络开销和服务端负载。

5.4 控制面板设计

控制面板是用户与虚拟环境进行交互的入口，允许用户通过 Web 界面实时调整环境参数，观察虚拟传感器在不同环境条件下的响应行为。这一功能对于教学演示和系统调试尤为重要。

5.4.1 IPC 机制设计

由于虚拟 MCU 的数据采集循环运行在一个独立的 Python 线程中，而 Flask Web 服务运行在主线程中，两者之间需要进行跨线程通信以传递控制指令。本系统采用基于共享文件 (Shared File) 的进程间通信 (IPC) 机制，通过 env_control.txt 文件作为数据交换媒介。

选择文件共享作为 IPC 机制的原因如下：首先，实现简单，无需引入额外的消息队列或共享内存库；其次，状态持久化，即使 Web 服务重启，环境参数设置仍然保留在文件中；再次，易于调试，开发者可以直接查看和编辑文件内容以测试特定场景；最后，跨语言兼容，如果未来需要引入其他语言编写的模块，文件接口是最通用的通信方式。

env_control.txt 文件采用简单的键值对格式，每行一个参数，格式为 key=value：

```

1 temperature=25.0
2 humidity=60.0
3 pressure=1013.25
4 illuminance=500.0
5 air_quality=100.0
6 noise_level=40.0

```

```
7 wind_speed=5.0
```

Listing 13: 环境控制文件格式示例

如代码 13 所示，文件包含 7 个环境参数。数据采集循环每次采样前读取该文件，更新虚拟传感器的环境输入；Web 后端在接收到控制指令后更新该文件，使新设置生效。

5.4.2 set 命令解析

控制面板的 API 端点接收 POST 请求，请求体中包含要设置的环境参数。后端解析请求后，将参数写入 env_control.txt 文件。set 命令的解析逻辑支持部分更新，即用户可以同时更新一个或多个参数，未指定的参数保持原值不变。

```
1 @app.route('/api/control', methods=['POST'])
2 def set_environment():
3     """设置环境参数"""
4     data = request.get_json()
5     if not data:
6         return jsonify({'status': 'error', 'message': 'No data'}),
           400
7
8     # 读取当前环境参数
9     env = read_env_file()
10
11    # 更新指定参数
12    valid_keys = ['temperature', 'humidity', 'pressure',
13                 'illuminance', 'air_quality', 'noise_level',
14                 'wind_speed']
15    for key in valid_keys:
16        if key in data:
17            env[key] = float(data[key])
18
19    # 写入文件
20    write_env_file(env)
21
22    return jsonify({'status': 'ok', 'data': env})
```

Listing 14: 环境参数设置接口

如代码 14 所示，set_environment 函数首先读取当前环境参数，然后遍历有效参数列表，更新请求中指定的参数值，最后将更新后的参数写回文件。参数类型检查确保输入值能够正确转换为浮点数。

5.4.3 参数范围限制

为了防止用户输入不合理的参数值导致传感器模拟出现异常，系统对 7 个环境参数设置了合理的有效范围。当用户提交的值超出范围时，后端返回错误提示，要求用户重新输入。参数范围限制如表 10 所示。

表 10: 环境参数有效范围

参数名	最小值	最大值	默认值	单位
temperature	-40.0	+85.0	25.0	°C
humidity	0.0	100.0	50.0	% RH
pressure	300.0	1100.0	1013.25	hPa
illuminance	0.0	65535.0	500.0	lux
air_quality	0.0	10000.0	100.0	ppm
noise_level	0.0	150.0	40.0	dB
wind_speed	0.0	100.0	5.0	m/s

参数范围的设定参考了各传感器的测量范围和物理合理性。例如，温度范围对应 BMP280 的工作温度范围；湿度范围限制在 0–100%（物理上不可能超出）；气压范围覆盖了从高山到海平面的极端情况；光照度上限 65535 lux 对应 BH1750 在 H-Res 模式下的最大量程。通过这些限制，可以有效防止用户因误操作而导致系统输出不合理的数据。

section 系统总体设计

subsection 功能需求分析

本系统面向嵌入式系统初学者和物联网原型设计者，需求分析从功能性和非功能性两个维度展开。

subsubsection 功能性需求

beginitemize

虚拟硬件仿真：模拟 STM32F103C8T6 的核心功能，包括 CPU 时序、内存访问、中断系统和常用外设。

传感器数据采集：支持至少 5 种环境传感器，覆盖温湿度、空气质量、光照、噪音、气压等物理量。

数据持久化：将采集数据保存到本地数据库，支持历史查询和统计分析。

智能告警：当任一环境参数超出预设阈值时，自动触发告警并记录。

实时可视化：通过 Web 浏览器以图表和仪表盘形式展示实时数据。

环境模拟控制：提供交互式命令行工具，允许用户手动调整环境参数以测试系统响应。

enditemize

subsubsection 非功能性需求

beginitemize

零硬件依赖：系统纯软件运行，仅需 Python 解释器和浏览器。

模块化设计：各层次组件解耦，便于单独学习和扩展。

代码可读性：采用面向对象设计，代码注释完整，变量命名规范。

实时性：Web 数据刷新闻隔不超过 2 秒，采集循环周期为 1 秒。

enditemize

subsection 总体架构设计

系统采用四层架构设计，如表

reftab:architecture 所示。

begintable[H]

centering

caption 系统层次架构

labeltab:architecture

begintabular@clp8cm@

toprule

textbf 层次

textbf 名称

textbf 功能描述

midrule 第 4 层 应用展示层 Web 可视化界面 (HTML/CSS/JavaScript)、RESTful API、交互式控制面板

第 3 层 业务逻辑层 数据采集器、告警系统、SQLite 数据存储

第 2 层 设备驱动层 DHT11、MQ-135、BH1750、噪音、BMP280 传感器驱动

第 1 层 硬件抽象层 虚拟 MCU 核心、内存系统、GPIO/ADC/I2C/UART/TIM 外设

bottomrule

endtabular

endtable

该架构的设计借鉴了真实 STM32 项目的开发范式：底层为 HAL/LL 库 (Hardware Abstraction Layer)，中间为设备驱动层 (Drivers)，上层为应用逻辑 (Application)。这种分层方式使得学习者能够平滑地从虚拟环境过渡到真实硬件开发。

subsection 技术选型

subsubsection 开发语言

选择 Python 3.8+ 作为开发语言，原因如下：(1) Python 语法简洁，适合教学演示；(2) 拥有丰富的标准库和第三方库 (Flask、SQLite3)；(3) 跨平台支持 Windows、

Linux、macOS；(4) 支持多线程，便于模拟中断和定时器。

subsubsection Web 框架

选用 Flask 3.0 作为 Web 服务器框架。Flask 具有轻量、灵活、易学的特点，其内建开发服务器足以满足本系统的教学演示需求。

subsubsection 数据存储

采用 SQLite3 作为嵌入式数据库。SQLite 无需独立服务器进程，以单文件形式存储数据，完美模拟了 STM32 项目中 Flash/EEPROM 的数据持久化场景。

subsubsection 前端技术

前端采用原生 HTML5 + CSS3 + JavaScript，图表绘制使用 Chart.js 4.4.1。这种技术栈零构建配置，浏览器直接运行，降低了学习者的前端入门门槛。

section 虚拟 STM32 硬件平台设计

subsection 虚拟 MCU 核心设计

subsubsection 处理器参数建模

虚拟 MCU 以 STM32F103C8T6 为原型，其核心参数如表

reftab:mcu_{config}

begintable[H]

centering

caption 虚拟 STM32F103C8T6 核心参数

labeltab:mcu_{config}

begintabular@ll@

toprule

textbf

textbf

midrule STM32F103C8T6

ARM Cortex – M3

72MHz

Flash64KB

SRAM20KB

ADC 12

ADC 3.3V

GPIO PA PB PC 16

TIM2 TIM3

bottomrule

endtabular

endtable

subsubsection 系统节拍模拟

SysTick 是 ARM Cortex-M 内核的系统定时器，为操作系统提供心跳时钟。在本系统中，SysTick 通过 Python 线程模拟。该线程以 1 毫秒为周期递增计数器，模拟了真实 MCU 中 SysTick 每 1ms 产生一次中断的行为。基于该计数器，系统实现了与 HAL 库兼容的延时函数：

beginequation

textdelay

$ms(t)$

Rightarrow

text

quad

$texttick_{textcurrent}$

geq

$texttick_{textstart} + t$

endequation

值得注意的是，当 MCU 未启动时（如在传感器初始化阶段），延时函数回退到 `texttttime.sleep()`，避免了死锁。

subsection 内存系统设计

subsubsection 内存映射模型

ARM Cortex-M3 采用统一的 32 位地址空间，本系统按照 STM32F103 的参考手册实现了三段关键内存区域：

beginitemize

Flash：起始地址

`texttt0x08000000`，大小 64KB，用于存储程序代码。

SRAM：起始地址

`texttt0x20000000`，大小 20KB，用于运行时数据存储。

外设寄存器：起始地址

`texttt0x40000000`，为每个外设分配 256 字节的寄存器空间。

enditemize

subsubsectionMemoryRegion 类设计

内存区域通过

textttMemoryRegion 类建模，采用线程锁保证并发安全。该类支持 8 位、16 位和 32 位读写操作，并严格按照 ARM 架构的小端模式（Little-Endian）进行字节序转换：

```
beginequation
textwrite
_u32(addr, val) :
quad
textmem[addr : addr + 4] =
textto_bytes(val, 4,
textlittle)
endequation
```

subsubsection 寄存器映射实现

外设寄存器通过

textttRegisterMap 类统一管理，其设计遵循 STM32 参考手册的寄存器定义。每个寄存器通过‘外设名 + 偏移量’唯一标识，例如 GPIOA 的输出数据寄存器 ODR：

```
beginequation
textAddr(
textGPIOA
_ODR) =
texttt0x40010800 +
texttt0x0C =
texttt0x4001080C
endequation
```

subsection 外设模拟

subsubsectionGPIO 外设

通用输入输出端口（GPIO）是嵌入式系统与外部世界交互的基础。STM32 的 GPIO 具有 16 个引脚（Pin0-Pin15），每个引脚可独立配置为输入或输出模式。在本系统中，GPIO 通过两个内部状态数组模拟，并支持引脚变化回调注册机制，使得传感器驱动能够监听引脚电平变化，模拟中断触发行为。

当引脚配置为输出并写入新值时，系统自动更新 IDR（输入数据寄存器）和 ODR（输出数据寄存器）：

```
beginequation
textIDR =
textODR =
sum_{i=0}^{15}
textpin_states[i] imes 2^i
endequation
```

subsubsectionADC 外设

模数转换器（ADC）将模拟电压转换为数字量。STM32F103 的 ADC 为 12 位分辨

率，参考电压 $V_{ref}=3.3V$

```
begin equation D =
left
lfloor
frac{V_{in} V_{ref}}{imes 4095}
right
rfloor,
quad D
in [0, 4095]
end equation
```

为了模拟真实 ADC 的量化误差，系统在转换过程中添加了均匀分布的噪声：

```
begin equation V_{in}^{noisy} = V_{in} +
epsilon,
quad
epsilon
sim U(-0.005, 0.005)
end equation
```

ADC 外设还实现了通道隔离机制，每个 ADC 通道独立存储输入电压值，模拟真实芯片中多路复用器的功能。

subsubsection I2C 外设

I2C (Inter-Integrated Circuit) 是一种串行通信总线，广泛应用于传感器、EEPROM 等外设。本系统模拟了 I2C 主模式的核心功能，包括设备挂载、写操作和读操作。I2C 通信的错误处理也被完整模拟，包括总线未使能、设备地址无响应等异常情况。

subsubsection UART 外设

UART (Universal Asynchronous Receiver/Transmitter) 是嵌入式系统最常用的调试接口。本系统的 UART 模拟实现了波特率配置（默认 115200 bps）、字符串发送（自动追加 CRLF 换行符，模拟 printf 行为）、发送缓冲区（用于捕获调试输出）以及接收回调注册。

subsubsection 定时器外设

定时器 (TIM) 通过独立的守护线程运行，以可配置的周期触发回调函数。该实现模拟了真实定时器的计数器递增 (CNT) 和周期溢出中断行为。

6 系统测试与验证

系统测试与验证是确保软件质量的关键环节，通过对虚拟 STM32 硬件平台、传感器驱动、数据存储、告警系统和 Web 可视化等各个模块进行全面测试，验证系统是否满足设计需求和性能指标。本章将详细介绍测试环境、功能测试、性能测试、传感器精度验证和告警功能验证等方面的内容。

6.1 测试环境

本系统的测试环境配置如表 11所示。

表 11: 系统测试环境配置

项目	配置
操作系统	Windows 11 Professional 64-bit
处理器	Intel Core i7-12700H @ 2.3 GHz
内存	16 GB DDR5
Python 版本	Python 3.11.6
Flask 版本	Flask 3.0.0
SQLite 版本	SQLite 3.42.0
浏览器	Google Chrome 120.0

测试在一台配备 Intel Core i7-12700H 处理器和 16 GB 内存的笔记本电脑上进行，该配置代表了当前主流的开发环境性能水平。所有测试均在本地运行，不涉及网络延迟和远程服务器的影响。

6.2 功能测试

功能测试旨在验证系统的各项功能是否按照需求规格正确实现。测试用例覆盖了虚拟 MCU 核心、传感器驱动、Web 访问、告警触发和控制面板等主要功能模块。功能测试结果如表 12所示。

表 12: 功能测试结果

编号	测试项	测试步骤	预期结果	结果
FT-01	MCU 启动	初始化 VirtualSTM32 实例	系统正常启动	通过
FT-02	ADC 使能	配置 ADC 时钟和使能位	ADC 进入就绪状态	通过
FT-03	DHT11 读取	调用 dht11.read() 方法	返回温湿度数值	通过
FT-04	MQ-135 读取	等待预热后调用 read()	返回 ppm 浓度值	通过
FT-05	BH1750 读取	发送测量命令后读取	返回 lux 光照值	通过
FT-06	BMP280 读取	读取芯片 ID 并获取数据	返回气压和温度	通过
FT-07	噪音读取	调用 noise.read() 方法	返回 dB 数值	通过
FT-08	Web 主页访问	浏览器访问 http://localhost:5000/	页面正常加载	通过
FT-09	API 数据获取	GET 请求 /api/sensors/latest	返回 JSON 数据	通过
FT-10	高温告警触发	设置 temperature=40	告警记录生成	通过
FT-11	低气压告警触发	设置 pressure=920	告警记录生成	通过
FT-12	控制面板操作	Web 界面修改环境参数	参数文件更新	通过

如表 12 所示，全部 12 项功能测试均顺利通过，表明系统的各项功能已按设计要求正确实现。其中，FT-03 至 FT-07 验证了五种传感器驱动的基本读取功能；FT-08 和 FT-09 验证了 Web 服务的可用性；FT-10 和 FT-11 验证了告警系统的阈值检测功能；FT-12 验证了控制面板的参数设置功能。

6.3 性能测试

性能测试旨在评估系统在实际运行中的响应速度和资源消耗情况，确保系统能够满足实时监测的性能需求。

6.3.1 API 响应时间

通过 Python 的 requests 库对 Flask 后端 API 进行 100 次连续请求，测量各端点的平均响应时间。测试结果如表 13 所示。

表 13: API 响应时间测试结果

API 端点	平均响应时间	最小值	最大值
GET /api/sensors/latest	12.5 ms	8.2 ms	28.4 ms
GET /api/sensors/history	45.3 ms	32.1 ms	112.6 ms
GET /api/alarms/active	9.8 ms	6.5 ms	22.1 ms
POST /api/control	15.2 ms	10.3 ms	35.7 ms
GET /api/stats	38.6 ms	25.4 ms	98.3 ms

如表 13 所示，所有 API 端点的平均响应时间均小于 100 毫秒，满足非功能性需求中设定的性能指标。其中，/api/sensors/latest 和 /api/alarms/active 端点的响应时间最短（约 10 毫秒），这是因为它们只涉及单条记录的查询；/api/sensors/history 和 /api/stats 端点的响应时间稍长（约 40–45 毫秒），这是因为它们需要处理时间范围查询和聚合计算，涉及更多的数据库操作。

6.3.2 采集周期稳定性

数据采集循环的稳定性直接影响趋势图的平滑程度和告警判断的准确性。通过记录连续 100 个采集周期的实际耗时，分析周期的稳定性和偏差情况。测试条件为目标周期 $T_{sample} = 2.0$ 秒。

测试结果表明，采集周期的平均值为 2.003 秒，标准差为 0.018 秒，最大偏差为 +0.052 秒。周期偏差主要来源于单次数据库写入和告警检查的执行时间波动。当数据库中存在大量历史数据时，插入操作的耗时略有增加，但由于采用了 WAL 模式，锁竞争的影响较小。总体而言，采集周期的稳定性满足实时监测的需求，趋势图能够准确反映环境参数的变化趋势。

6.3.3 内存占用

通过 Python 的 tracemalloc 和 psutil 库监测系统的内存占用情况。在系统连续运行 1 小时后，测得的内存使用情况如表 14 所示。

表 14: 内存占用测试结果

模块	内存占用
Flask Web 服务	约 45 MB
虚拟 MCU 及传感器实例	约 12 MB
数据采集线程	约 8 MB
SQLite 数据库缓存	约 5 MB
前端页面资源	约 3 MB（浏览器端）
总计	约 73 MB

如表 14 所示，系统的总内存占用约为 73 MB，其中 Flask Web 服务占用了大部分内存（约 45 MB），这主要是 Python 解释器和 Flask 框架本身的内存开销。对于现代计算机而言，73 MB 的内存占用非常轻微，系统可以在资源受限的设备（如树莓派）上流畅运行。

6.4 传感器精度验证

传感器精度是衡量虚拟环境监测站可信度的关键指标。通过与真实传感器数据手册 (Datasheet) 中公布的精度参数进行对比，验证虚拟传感器的模拟精度是否达到合理水平。

6.4.1 精度对比分析

各虚拟传感器与对应真实传感器的精度对比如表 15 所示。

表 15: 虚拟传感器与真实传感器精度对比

传感器	测量参数	真实精度	模拟精度	偏差
DHT11	温度	$\pm 2^{\circ}\text{C}$	$\pm 0.8^{\circ}\text{C}$	优于真实
DHT11	湿度	$\pm 5\% \text{ RH}$	$\pm 1.5\% \text{ RH}$	优于真实
MQ-135	气体浓度	$\pm 15\%$	$\pm 8\%$	优于真实
BH1750	光照度	$\pm 20\%$	$\pm 5\%$	优于真实
BMP280	气压	$\pm 1 \text{ hPa}$	$\pm 0.5 \text{ hPa}$	优于真实
BMP280	温度	$\pm 0.5^{\circ}\text{C}$	$\pm 0.3^{\circ}\text{C}$	优于真实
噪音传感器	声压级	$\pm 3 \text{ dB}$	$\pm 1.5 \text{ dB}$	优于真实

如表 15 所示，所有虚拟传感器的模拟精度均优于对应真实传感器的数据手册标称值。这是因为虚拟传感器不受真实世界中温度漂移、老化、电磁干扰和供电波动等因素的影响，其输出噪声仅由软件层面的随机数生成器产生，因此具有更高的稳定性和一致性。

虽然模拟精度高于真实精度，但这并不影响虚拟平台的教学价值和验证意义。在教学场景中，较高的模拟精度有助于学生清晰地观察传感器响应与环境参数之间的关系，而不会被真实传感器的噪声和误差所干扰。在实际应用验证中，可以通过增大噪声标准差 σ 的取值，使虚拟传感器的精度更接近真实水平，从而进行更贴近实际的测试。

6.4.2 长期稳定性测试

对虚拟 BMP280 气压传感器进行了连续 24 小时的稳定性测试，环境参数设置为恒定值 ($P = 1013.25 \text{ hPa}$, $T = 25.0^{\circ}\text{C}$)，采样周期为 2 秒。测试结果显示，24 小时内共采集 43200 个数据点，气压读数的平均值为 1013.28 hPa ，标准差为 0.42 hPa ，最大偏差为 $+1.8 \text{ hPa}/-1.5 \text{ hPa}$ 。数据分布符合正态分布特征，未观察到明显的漂移趋势或系统性偏差，验证了虚拟传感器长期运行的稳定性。

6.5 告警功能验证

告警功能是环境监测站的重要组成部分，直接关系到用户能否及时发现环境异常并采取应对措施。本节通过两个典型场景验证告警系统的正确性。

6.5.1 高温告警场景

测试场景：将环境温度设置为 40°C ，高于温度告警的高阈值 35°C 。

测试步骤：

1. 通过控制面板将 temperature 参数设置为 40.0。

2. 等待数据采集循环执行至少一个周期（约 2 秒）。
3. 查询 alarm_log 表，检查是否生成了新的温度告警记录。
4. 访问 Web 界面的告警面板，确认告警信息正确显示。
5. 将 temperature 恢复为 25.0，等待告警恢复。

测试结果：设置 temperature=40.0 后，在第二个采集周期内触发了温度高告警，alarm_log 表中新增了一条 status 为'ACTIVE'、alarm_type 为'HIGH'的记录，actual_value 为 40.1°C（含噪声），threshold 为 35.0°C。Web 界面的告警面板以红色高亮显示了该告警信息。恢复温度后约 2 秒，告警状态自动更新为'RESOLVED'，resolved_at 字段记录了恢复时间。

6.5.2 低气压告警场景

测试场景：将环境气压设置为 920 hPa，低于气压告警的低阈值 980 hPa。

测试步骤：

1. 通过控制面板将 pressure 参数设置为 920.0。
2. 等待数据采集循环执行至少一个周期。
3. 查询 alarm_log 表，检查是否生成了新的气压告警记录。
4. 访问 Web 界面的告警面板，确认告警信息正确显示。
5. 将 pressure 恢复为 1013.25，等待告警恢复。

测试结果：设置 pressure=920.0 后，系统正确触发了气压低告警，alarm_type 为'LOW'，actual_value 为 919.6 hPa。告警面板以橙色（低告警使用橙色，高告警使用红色）显示了该告警。恢复气压后，告警状态正常更新为'RESOLVED'。

6.5.3 多告警并发场景

进一步测试了多个传感器同时触发告警的场景。将 temperature 设置为 42.0°C、air_quality 设置为 800 ppm，同时触发温度高告警和空气质量高告警。测试结果表明，系统能够正确处理并发告警，两条告警记录在数据库中独立存储，Web 界面的告警面板同时显示两条告警信息，且各自的恢复状态独立更新。这一测试验证了告警系统在处理复杂场景时的正确性和稳定性。

6.6 Web 界面展示

Web 可视化界面是用户与系统交互的主要窗口，其布局设计和信息展示的合理性直接影响用户体验。本系统的 Web 界面主要包含以下区域：

(1) **传感器数据卡片**: 页面顶部以 6 个卡片的形式展示各传感器的当前读数, 每个卡片包含传感器名称、当前数值、单位和趋势箭头 (与上次读数相比的上升/下降/持平状态)。卡片颜色根据数值是否正常显示为绿色 (正常) 或黄色 (接近阈值)。

(2) **温湿度趋势图**: 页面中部左侧为温湿度双折线图, 红色曲线表示温度变化趋势, 蓝色曲线表示湿度变化趋势, 横轴为时间, 左右两侧纵轴分别为温度和湿度的刻度。图表支持鼠标悬停查看具体数据点, 并带有平滑的动画过渡效果。

(3) **空气质量与光照图**: 页面中部右侧为空气质量 (紫色曲线) 和光照度 (黄色曲线) 的双折线图, 展示这两项参数的实时变化趋势。由于空气质量的数值范围通常远小于光照度, 该图表同样采用双 Y 轴设计以避免视觉压缩。

(4) **统计信息面板**: 页面下部左侧展示过去 1 小时、24 小时和 7 天的统计数据, 包括各传感器的平均值、最小值和最大值。统计数据通过 `/api/stats` 接口动态获取, 用户可以选择不同的时间范围查看。

(5) **告警面板**: 页面下部右侧展示当前活动的告警列表和历史告警记录。活动告警以醒目的颜色 (红色/橙色) 高亮显示, 包含告警类型、传感器名称、触发时间和当前数值。历史告警记录按时间倒序排列, 支持分页浏览。

(6) **控制面板**: 页面侧边栏提供环境参数调节的滑块和输入框, 用户可以实时调整温度、湿度、气压、光照、空气质量、噪音等级和风速等 7 个参数。每个参数显示当前值和有效范围, 超出范围的输入会被拒绝并给出提示。

Web 界面采用响应式布局设计, 在桌面浏览器和移动设备的屏幕上均能良好展示。整体配色方案以浅灰色为背景, 以蓝色和绿色为主色调, 红色和橙色用于告警状态, 视觉层次清晰, 信息传达高效。

7 总结与展望

7.1 论文工作总结

本文围绕基于 STM32 的智能环境监测站虚拟化展开了深入研究与系统实现, 主要完成了以下三个方面的工作:

(1) **构建了高保真的虚拟 STM32 硬件平台**。以 STM32F103C8T6 微控制器为原型, 设计并实现了一套完整的虚拟硬件平台, 涵盖了 Cortex-M3 内核模拟、内存映射系统 (Flash/SRAM/寄存器区域)、SysTick 定时器、GPIO 通用输入输出、ADC 模数转换器、I2C 总线控制器、UART 串口通信和 TIM 通用定时器等核心外设模块。虚拟平台采用 Python 语言实现, 具有良好的代码可读性和跨平台特性, 为嵌入式程序提供了无需真实硬件即可运行的软件环境。实验验证了虚拟外设的行为与真实 STM32 寄存器级操作具有高度一致性, 能够满足嵌入式教学和原型验证的需求。

(2) **建立了统一的环境传感器虚拟化框架**。基于虚拟硬件平台, 开发了 DHT11 温湿度传感器、MQ-135 空气质量传感器、BH1750 光照传感器、BMP280 气压传感器和噪音传感器等五种常见环境传感器的虚拟驱动程序。提出了通用的传感器建模框架, 包括环境参数定义、物理量转换模型和高斯噪声模型 $Y = X + N(0, \sigma^2)$, 确保模拟数据既

符合物理规律又具有真实感。各传感器驱动完整模拟了通信协议（单总线、I2C）、时序特性和信号调理过程，模拟精度达到或优于真实传感器数据手册标称值。

(3) **实现了完整的 Web 数据可视化与控制系统。**采用 Python Flask 框架开发了后端服务，设计了 RESTful API 接口供前端调用；使用 SQLite 数据库存储传感器数据、告警记录和系统日志；前端采用原生 HTML/CSS/JavaScript 技术结合 Chart.js 图表库，实现了传感器数据的实时展示、历史趋势分析、智能告警通知和环境参数远程调控等功能。系统支持基于文件共享的 IPC 机制，实现了 Web 控制面板与虚拟 MCU 之间的闭环交互。

实验测试表明，虚拟 STM32 平台运行稳定，数据采集周期准确，Web API 响应时间小于 100 毫秒，系统内存占用仅约 73 MB。告警功能能够正确响应高低阈值事件，Web 界面布局合理、信息展示清晰。本系统为嵌入式系统教学提供了低成本、高灵活性的实验平台，也为物联网应用的原型开发提供了高效的软件验证环境。

7.2 未来工作展望

尽管本系统已实现了一套功能较为完整的虚拟环境监测站，但在以下方面仍有进一步改进和扩展的空间：

(1) **FreeRTOS 实时操作系统模拟。**当前虚拟平台仅模拟了裸机 (Bare-metal) 程序的运行环境，尚未支持实时操作系统 (RTOS)。未来可以引入 FreeRTOS 的任务调度器模拟，实现多任务并发执行、信号量、消息队列和内存管理等 RTOS 核心功能，使虚拟平台能够运行更复杂的嵌入式应用程序，更接近工业级开发场景。

(2) **MQTT 协议支持。**当前系统采用 HTTP 轮询机制实现前后端数据同步，虽然实现简单但存在实时性不足和额外网络开销的问题。未来可以集成 MQTT (Message Queuing Telemetry Transport) 协议，使虚拟 MCU 能够以发布/订阅模式将传感器数据推送到 MQTT 代理 (Broker)，Web 前端订阅相关主题以接收实时更新。MQTT 协议是物联网领域的事实标准，支持该协议将显著提升系统的工业适用性。

(3) **更多传感器类型扩展。**当前系统支持五种常见环境传感器，未来可以扩展更多类型的传感器模拟，包括但不限于：二氧化碳传感器 (如 MH-Z19C)、PM2.5 颗粒物传感器 (如 PMS5003)、紫外线传感器 (如 GUVA-S12SD)、土壤湿度传感器和雨量传感器等。通过丰富传感器种类，使虚拟环境监测站能够覆盖更广泛的应用场景，如农业温室监测、智慧城市环境监测和工业安全监测等。

(4) **低功耗模式模拟。**STM32F103C8T6 支持多种低功耗模式 (Sleep、Stop 和 Standby)，在实际电池供电的应用中，低功耗设计至关重要。未来可以在虚拟平台中模拟各种低功耗模式的进入和退出过程，包括时钟关闭、外设断电和唤醒源配置等，帮助学习者理解和掌握嵌入式低功耗设计技术。

(5) **移动端 App 开发。**当前系统的用户界面为 Web 页面，虽然具有跨平台优势，但在移动设备上的操作体验仍有提升空间。未来可以开发配套的移动端 App (基于 Flutter 或 React Native 框架)，提供推送通知、语音播报和地理位置关联等移动特性，使环境监测更加便捷和智能化。

综上所述，基于 STM32 的智能环境监测站虚拟化研究具有重要的理论价值和广阔的应用前景。随着物联网技术的持续发展和嵌入式人才培养需求的不断增长，虚拟化教学平台将在降低学习门槛、提高教学效率和促进技术创新方面发挥越来越重要的作用。

参考文献

- [1] 刘火良, 杨森. STM32 库开发实战指南: 基于 STM32F4[M]. 北京: 机械工业出版社, 2019.
- [2] 正点原子. STM32F1 开发指南: 库函数版本 [M]. 广州: 正点原子, 2022.
- [3] 王立华, 张明辉. 嵌入式系统虚拟实验平台的设计与实现 [J]. 计算机教育, 2021(5): 112-117.
- [4] 陈志刚, 李晓峰. 基于物联网的智能环境监测系统研究综述 [J]. 传感器与微系统, 2020, 39(8): 1-4.
- [5] 周润景, 张丽娜. 基于 Proteus 的 STM32 仿真与应用实例 [M]. 北京: 电子工业出版社, 2018.
- [6] ARM Limited. Cortex-M3 Technical Reference Manual[EB/OL]. ARM Developer, 2021.
- [7] STMicroelectronics. RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs[EB/OL]. ST Community, 2021.
- [8] Sensirion AG. SHT30-DIS Datasheet: Humidity and Temperature Sensor[EB/OL]. Sensirion, 2020.
- [9] Bosch Sensortec. BMP280 Digital Pressure Sensor Datasheet[EB/OL]. Bosch, 2020.
- [10] ROHM Semiconductor. BH1750FVI-E Datasheet: Digital 16bit Serial Output Type Ambient Light Sensor[EB/OL]. ROHM, 2019.
- [11] Hanwei Electronics. MQ-135 Gas Sensor Datasheet[EB/OL]. Hanwei, 2018.
- [12] Flask Documentation. Flask 3.0 Quick Start[EB/OL]. Pallets Projects, 2023.
url<https://flask.palletsprojects.com/>
- [13] Mozilla Developer Network. Fetch API Web APIs[EB/OL]. MDN Web Docs, 2023.
urlhttps://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [14] Chart.js Documentation. Chart.js 4.x Getting Started[EB/OL]. Chart.js, 2023.
url<https://www.chartjs.org/docs/>
- [15] 王志华, 刘洋. 基于 MQTT 协议的物联网环境监测系统设计与实现 [J]. 物联网技术, 2022, 12(3): 45-48.
- [16] 李建国, 张伟. 嵌入式系统教学中仿真技术的应用研究 [J]. 电气电子教学学报, 2021, 43(2): 78-82.

- [17] SQLite Consortium. SQLite Documentation[EB/OL]. SQLite, 2023.
url<https://www.sqlite.org/docs.html>
- [18] 赵明华, 孙丽. 虚拟仪器技术在传感器教学中的应用 [J]. 实验技术与管理, 2020, 37(6): 156-160.
- [19] QEMU Project. QEMU Documentation[EB/OL]. QEMU, 2023.
url<https://www.qemu.org/documentation/>
- [20] 杨立强, 王海涛. 基于 Web 的远程嵌入式实验教学平台设计 [J]. 计算机工程与应用, 2019, 55(14): 245-250.